

Danmarks
Tekniske
Universitet



A From-Scratch Neural Network for Binary Classification

Base Python and NumPy Implementations

22118 Advanced Unix & Python for Bioinformaticians

Repository: https://github.com/andregodinhodtu/ANN_Backprop_Implement

Team Members
André Godinho - s253707
Lun Suša - s253705

May 8, 2026

Contents

1	Introduction	4
1.1	Importance of ANNs	4
1.2	ANNs in bioinformatics	4
1.3	ANNs for prediction from SNPs	4
1.4	This project	5
2	Contribution	6
3	Theory	7
3.1	Feedforward Neural Network	7
3.1.1	Input Vector	7
3.1.2	Weights and Biases	7
3.1.3	Forward Pass Through a Layer	7
3.1.4	Activation Functions	7
3.1.5	Network Architecture	8
3.1.6	Parameter Initialization	9
3.2	Training neural networks	9
3.2.1	Loss Function	9
3.2.2	Gradient Descent	9
3.2.3	Backpropagation	10
3.2.4	L2 Regularization	12
3.2.5	Training Procedure	13
4	Algorithm Design	14
4.1	Layer Initialisation	14
4.2	Forward Pass	15
4.2.1	Single Sample	15
4.2.2	Batch Processing	15
4.3	Loss Functions	16
4.4	Backpropagation: Error Delta Computation	16
4.5	Gradient Computation	17
4.6	Parameter Update with L2 Regularisation	18
4.7	Training Strategy	18
4.7.1	Class Imbalance — Oversampling	18
4.7.2	Training Loop: Mini-Batch Gradient Descent with Early Stopping and Learning-Rate Decay	19
4.8	Evaluation	21
4.8.1	Accuracy	21
4.8.2	Confusion Matrix	21
4.8.3	Classification Metrics	22
4.8.4	Evaluate and Report	22
4.9	Data Parsing	23
4.10	Summary of Base Python vs NumPy Differences	24

5	Program Design	26
5.1	Repository Structure	26
5.2	Files and Responsibilities	26
5.3	Program Flow	27
6	Program Manual	29
6.1	Scope and limitations	29
6.2	Running the example scripts	29
6.3	Input format	30
6.4	Train mode	30
6.5	Test mode	31
6.6	Context-dependent components	32
6.7	Configuration options	32
6.8	Saving and loading models	33
6.9	Expected output	33
6.10	Adapting the example scripts	33
7	Results	35
7.1	Experimental Setup	35
7.2	Training Convergence	36
7.3	Classification Results	36
7.3.1	Confusion Matrices	37
7.4	Empirical Runtime	37
8	Runtime Analysis	38
8.1	ANN Layer	38
8.1.1	$O(1)$ Operations	38
8.1.2	Weight and Bias Initialisation, Matrix Operations	38
8.1.3	Forward Pass and Derivatives	41
8.1.4	Overall Layer Complexity	42
8.2	ANN	42
8.2.1	$O(1)$ Functions	43
8.2.2	Network-Level Functions	43
8.2.3	ANN Training and Evaluation	46
8.2.4	Overall ANN Complexity	50
8.3	Big O of Evaluation	51
8.3.1	Overall Evaluation Complexity	53
8.4	Big O of Data Parsing	53
8.4.1	Overall Data Parsing Complexity	55
8.5	Big O of main	55
8.5.1	Overall main Complexity	57
9	Unit Test	58
9.1	Coverage	58
9.2	Types of evaluations	59

9.3	Round-trip testing for I/O	59
9.4	Testing gaps	59
10	Conclusion	60
10.1	Methodological caveats and known limitations	60
10.2	Future work	61
11	Declaration of use of generative AI	63
	References	64

1 Introduction

1.1 Importance of ANNs

ANN is a computing system resembling the basic structure of neurons in human (and other animals') brain. It consists of highly interconnected network of processing units called neurons, and has the ability to handle complex data features to process their information. It is an automated process, excelling at pattern recognition. [1]

The neurons are organised in layers, with connections from one layer feeding information into the next. Input layer accepts input information, the signal then travels through hidden layers to the last, output layer. The output of each neuron is computed by an activation function of the totality of its inputs. Signal strength at each connection is determined by weights, which adjust during the learning process [2,3]). With their ability to process data non-linearly, ANNs excel at finding patterns in large and complex datasets, that would be impossible to analyze manually or with purely linear models.

The performance of an ANN is evaluated by classification accuracy. A very important step for the success of a neural network is the parameter adjustment made by a feedback process called backpropagation. Weights and biases are modified from the output towards the input layer with the purpose of reducing the overall computational loss of the network. [1]

1.2 ANNs in bioinformatics

ANNs have been used in many areas of bioinformatics and are proven to be one of the most powerful tools in solving many problems, as they are able to process and analyze vast medical datasets. They enhance diagnostic accuracy by interpreting complex medical imaging for early disease detection and by predicting patient outcomes, and are efficient for use in personalized medicine. [2]

The relationships between genetic markers and traits or disease are usually noisy and complex, due to the interaction of many genetic and environmental factors. Such problems should be addressed by universal approximators of complex functions, which ANNs certainly are.

DTU HealthTech already runs several ANN-based prediction servers, which further proves the usefulness of this tool in this field.

1.3 ANNs for prediction from SNPs

Single nucleotide polymorphisms are single base sequence variations between individuals at a specific position in the genome, primarily responsible for phenotypic differences. They are suggested to affect the development of diseases in humans and the response to drug treatment and environmental stress. [4, 5]

The relationship between SNPs and phenotypes is noisy and cryptic, due to the influence of many genetic and environmental factors on complex traits. As such, a perfect problem for the application of ANNs, for both quantitative and discrete traits. [5]

1.4 This project

The main purpose of this project is to develop an ANN, that would predict whether certain variations of a SNP will lead to disease or not, as efficiently and accurately as possible.

The data comes from DTU HealthTech and has been prepared for direct network training. 3 of the subsets are used in training, and the remaining one for testing of the tool.

For full transparency and clarity of the coding process, the first implementation of the ANN is done purely in base Python, followed by a more computationally efficient implementation using the NumPy library.

Both models learn using the backpropagation algorithm and their performance is evaluated based on classification accuracy.

2 Contribution

Both authors collaborated closely and contributed equally to the project.

3 Theory

3.1 Feedforward Neural Network

The implementation presented in this project is a classical **Feedforward Neural Network** (FNN), also known as a Multilayer Perceptron (MLP) [2,3,6]. In a feedforward network, information flows strictly in one direction, from the input layer, through one or more hidden layers, and finally to the output layer, with no cycles or feedback connections.

3.1.1 Input Vector

The network receives an input vector $\mathbf{x} \in \mathbb{R}^n$, where n is the number of input features. Each element x_i represents one feature of the data sample being processed.

3.1.2 Weights and Biases

Each connection between neurons is associated with a **weight** w_{ij} , which controls the strength and direction of the signal passed forward. Neurons in the hidden and output layers may also include a **bias** term b_i , which allows the activation to be shifted independently of the input. Together, weights and biases are the learnable parameters of the network [2,3].

3.1.3 Forward Pass Through a Layer

Between the input and output layers, the network can contain **hidden layers**. Each hidden layer l transforms its input using a weight matrix $\mathbf{W}^{(l)}$ and a bias vector $\mathbf{b}^{(l)}$. It is important to distinguish between two quantities computed at each layer [3,6]:

- The **pre-activation** $\mathbf{z}^{(l)} \in \mathbb{R}^{n_l}$, which is the raw linear combination of the previous layer's output:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (1)$$

- The **post-activation** $\mathbf{a}^{(l)} \in \mathbb{R}^{n_l}$, which is obtained by applying the activation function $f(\cdot)$ element-wise to each neuron's pre-activation:

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) \quad (2)$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ is the input vector and n_l denotes the number of neurons in layer l . Both $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$ are therefore vectors of length n_l , with one entry per neuron. The post-activation $\mathbf{a}^{(l)}$ is then passed as input to the next layer, propagating the signal forward through the network.

3.1.4 Activation Functions

The activation function $f(\cdot)$ introduces non-linearity into the network, which is essential for the model to learn complex patterns beyond simple linear relationships [2,3]. Without non-linear activations, stacking multiple layers would collapse into a single linear transformation,

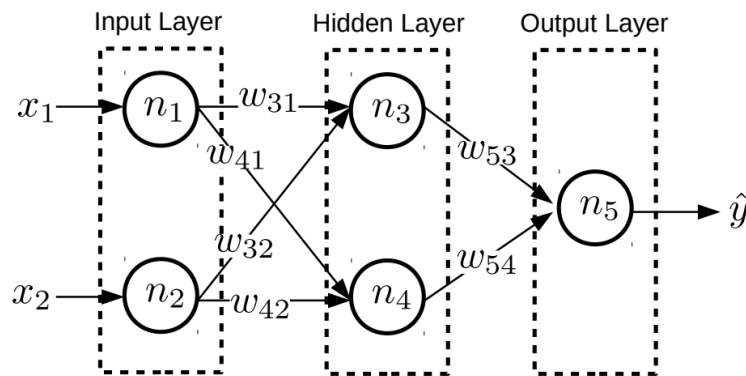


Figure 1: Simple neural network with 2 inputs, one hidden layer with 2 neurons, and a single output, connected by 6 weights [3].

regardless of depth. Common choices include the Rectified Linear Unit (*ReLU*) and the sigmoid function σ , defined respectively as:

$$\text{ReLU}(z) = \max(0, z) \quad (3)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (4)$$

The sigmoid function squashes its input to the range $(0, 1)$, which makes it well suited for output layers in binary classification tasks.

3.1.5 Network Architecture

The term **architecture** refers to the set of structural choices that define the network before training begins [3, 6]. These include:

- **Number of hidden layers:** deeper networks can model more complex functions, but are also harder to train and more prone to overfitting.
- **Number of neurons per layer:** wider layers increase the representational capacity of the network, at the cost of more parameters.
- **Choice of activation function:** different activations have different properties regarding computational cost, and suitability for different tasks.

In the classical training setting, these hyperparameters are not learned but are fixed design decisions made prior to training, and they significantly affect the network's capacity and behaviour.

3.1.6 Parameter Initialization

Before training begins, the weights and biases of every layer must be assigned initial values [2,3]. The choice of initialization scheme has a direct impact on whether the network can learn at all, and on how quickly it converges.

Biases are typically initialized to zero, since they only shift activations and there is no symmetry concern associated with them.

Regarding weights, a simple approach is to draw each from a uniform or normal distribution centred at zero with a small standard deviation. More sophisticated schemes such as Xavier or He initialization scale the variance based on the number of input and output neurons of each layer, which helps keep the magnitude of the activations and gradients stable across layers, particularly in deeper networks [7,8].

3.2 Training neural networks

Once the architecture of a feedforward network has been fixed and its parameters initialized, training consists of finding values for the weights and biases that allow the network to map inputs to the desired outputs as accurately as possible [3,6]. This is achieved by defining a **loss function** that quantifies the prediction error and then iteratively adjusting the parameters to minimize it.

3.2.1 Loss Function

Given a training sample with input $\mathbf{x}$ and target output $\mathbf{y}$, the loss function L measures how far the network's prediction $\mathbf{a}^{(L)}$ deviates from the desired output, where L denotes the index of the output layer. The choice of loss function depends on the task being solved [2,3].

A common choice for regression tasks is the **Mean Squared Error** (MSE):

$$L_{\text{MSE}} = \frac{1}{2} \|\mathbf{a}^{(L)} - \mathbf{y}\|^2 \quad (5)$$

The factor of $1/2$ is included for convenience, as it simplifies the derivative.

For binary classification tasks with a single output neuron, where the target $y \in \{0, 1\}$ and the output uses a sigmoid activation so that $a^{(L)} \in (0, 1)$ can be interpreted as a probability, a more suitable choice is the **Binary Cross-Entropy** (BCE) loss:

$$L_{\text{BCE}} = - \left[y \log(a^{(L)}) + (1 - y) \log(1 - a^{(L)}) \right] \quad (6)$$

This loss penalizes confident wrong predictions much more heavily than MSE: as $a^{(L)}$ approaches the wrong class, the logarithm diverges to $-\infty$, producing a large gradient that pushes the network to correct the prediction quickly.

3.2.2 Gradient Descent

Training proceeds by **gradient descent**: at each step, every parameter θ in the network is updated in the direction that decreases the loss most rapidly [2,3,6]:

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta} \quad (7)$$

where $\eta > 0$ is the **learning rate**, a hyperparameter that controls the step size. A value too small leads to slow convergence, while a value too large can cause oscillation or even divergence.

3.2.3 Backpropagation

Computing $\partial L / \partial \theta$ for every weight and bias in a deep network is made tractable by **backpropagation**, which applies the chain rule to propagate the error signal backward through the layers [3, 6, 9, 10].

For each layer l , an **error term** $\delta^{(l)} \in \mathbb{R}^{n_l}$ is defined as the gradient of the loss with respect to the pre-activation $\mathbf{z}^{(l)}$:

$$\delta_i^{(l)} = \frac{\partial L}{\partial z_i^{(l)}} \quad (8)$$

This quantity captures how sensitive the loss is to small changes in the pre-activation of neuron i in layer l , and it acts as the bridge between the loss and the parameter gradients.

Output layer. At the output layer L , the loss depends on $\mathbf{z}^{(L)}$ only through the post-activation $\mathbf{a}^{(L)} = f(\mathbf{z}^{(L)})$. Applying the chain rule component-wise [6, 9]:

$$\delta_i^{(L)} = \frac{\partial L}{\partial z_i^{(L)}} = \frac{\partial L}{\partial a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} = \frac{\partial L}{\partial a_i^{(L)}} \cdot f'(z_i^{(L)}) \quad (9)$$

which in vector form gives:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} L \odot f'(\mathbf{z}^{(L)}) \quad (10)$$

where $\odot$ denotes the element-wise product.

Derivatives of the activation functions. The element-wise term $f'(\mathbf{z}^{(l)})$ depends on which activation is used in layer l . For the activations implemented in this project, the derivatives are:

$$\text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \quad (11)$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (12)$$

Loss gradient for MSE. For the MSE loss in Equation 5, so differentiating with respect to each output gives:

$$\frac{\partial L_{\text{MSE}}}{\partial a_i^{(L)}} = a_i^{(L)} - y_i \implies \nabla_{\mathbf{a}^{(L)}} L_{\text{MSE}} = \mathbf{a}^{(L)} - \mathbf{y} \quad (13)$$

The output-layer error then becomes:

$$\delta^{(L)} = (\mathbf{a}^{(L)} - \mathbf{y}) \odot f'(\mathbf{z}^{(L)}) \quad (14)$$

Loss gradient for BCE with sigmoid output. For the BCE loss in Equation 6, differentiating with respect to the output yields:

$$\frac{\partial L_{\text{BCE}}}{\partial a^{(L)}} = -\frac{y}{a^{(L)}} + \frac{1-y}{1-a^{(L)}} = \frac{a^{(L)} - y}{a^{(L)}(1-a^{(L)})} \quad (15)$$

When BCE is paired with a sigmoid output activation, the activation derivative is $\sigma'(z^{(L)}) = a^{(L)}(1-a^{(L)})$, and substituting both expressions into Equation 10 produces a clean cancellation:

$$\delta^{(L)} = \frac{a^{(L)} - y}{a^{(L)}(1-a^{(L)})} \cdot a^{(L)}(1-a^{(L)}) = a^{(L)} - y \quad (16)$$

This cancellation is one of the main reasons BCE and sigmoid are commonly used together for binary classification: the resulting expression is numerically stable and identical in form to the MSE case, even though the underlying loss is different.

Hidden layers. For a hidden layer $l < L$, the loss depends on $\mathbf{z}^{(l)}$ only through its influence on the next layer's pre-activations $\mathbf{z}^{(l+1)}$. Applying the chain rule with a sum over the neurons of layer $l+1$ [9, 10]:

$$\delta_i^{(l)} = \frac{\partial L}{\partial z_i^{(l)}} = \sum_{k=1}^{n_{l+1}} \frac{\partial L}{\partial z_k^{(l+1)}} \cdot \frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} = \sum_{k=1}^{n_{l+1}} \delta_k^{(l+1)} \cdot \frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} \quad (17)$$

The remaining partial derivative follows from the definition $z_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} f(z_j^{(l)}) + b_k^{(l+1)}$, where $f(z_j^{(l)}) = a_j^{(l)}$ is the activation of neuron j in layer l , which gives:

$$\frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} = w_{ki}^{(l+1)} f'(z_i^{(l)}) \quad (18)$$

Substituting back:

$$\delta_i^{(l)} = f'(z_i^{(l)}) \sum_{k=1}^{n_{l+1}} w_{ki}^{(l+1)} \delta_k^{(l+1)} \quad (19)$$

which in vector form is:

$$\delta^{(l)} = (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot f'(\mathbf{z}^{(l)}) \quad (20)$$

The error at neuron i of layer l is the weighted sum of the errors it caused in the next layer, scaled by how strongly each next-layer neuron is connected to it ($w_{ki}^{(l+1)}$), and modulated by how reactive that neuron's activation is at its current pre-activation ($f'(z_i^{(l)})$). The transpose

of the weight matrix appears naturally because the same weights that propagate the signal forward also propagate the error backward.

Parameter gradients. Once the deltas are known, the gradients of the loss with respect to the weights and biases of layer l follow directly. Since $z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$, the chain rule gives $\partial z_i^{(l)} / \partial w_{ij}^{(l)} = a_j^{(l-1)}$ and $\partial z_i^{(l)} / \partial b_i^{(l)} = 1$, so:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top \quad (21)$$

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \quad (22)$$

Averaging over a batch. The gradients derived in Equations 21 and 22 correspond to a single training sample. In practice, networks are rarely trained on one sample at a time. Instead, a batch of B samples is processed together, and the **batch loss** $\mathcal{L}$ is defined as the average of the per-sample losses [3, 10]:

$$\mathcal{L} = \frac{1}{B} \sum_{s=1}^B L_s \quad (23)$$

where L_s denotes the loss evaluated on the s -th sample of the batch, using either Equation 5 or 6. Because differentiation is linear, the gradient of the batch loss with respect to any parameter is simply the average of the per-sample gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{1}{B} \sum_{s=1}^B \boldsymbol{\delta}_s^{(l)} (\mathbf{a}_s^{(l-1)})^\top \quad (24)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \frac{1}{B} \sum_{s=1}^B \boldsymbol{\delta}_s^{(l)} \quad (25)$$

where the subscript s indicates the per-sample quantity obtained from the forward and backward passes for sample s . In practice, each parameter update therefore involves running a full forward and backward pass for every sample in the batch, accumulating the resulting per-sample gradients, and dividing by B before applying the update rule in Equation 7.

3.2.4 L2 Regularization

To reduce overfitting, an **L2 regularization** term can be added to the batch loss, penalizing large weight values [2, 3]:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \sum_l \|\mathbf{W}^{(l)}\|_F^2 \quad (26)$$

where $\lambda \geq 0$ is the regularization coefficient and $\|\cdot\|_F$ denotes the Frobenius norm. The resulting weight update rule becomes:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \left(\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} + \lambda \mathbf{W}^{(l)} \right) \quad (27)$$

Biases are typically excluded from regularization, as they only shift activations and do not contribute to overfitting in the same way as weights:

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} \quad (28)$$

3.2.5 Training Procedure

A full pass over the training dataset is called an **epoch**. During each epoch, the network performs a forward pass to compute predictions, evaluates the loss, and runs backpropagation to update the parameters [3]. This is repeated for a fixed number of epochs, or until the loss converges.

A key design choice is the batch size B in Equations 24 and 25, i.e. how many samples are processed per update. Two main approaches exist [2, 3]:

- **Full-batch gradient descent:** B equals the entire training set. This yields the most accurate gradient estimate and smooth convergence, but each update is expensive and scales poorly.
- **Mini-batch gradient descent:** a compromise with B typically between 16 and 256. It balances stability and efficiency, introduces useful regularizing noise, and exploits vectorized hardware. This is the standard choice in practice.

For mini-batch training, the data is shuffled at the start of each epoch to prevent the network from learning patterns tied to sample ordering.

The learning rate η , regularization coefficient λ , batch size B , and number of epochs are hyperparameters that must be tuned to balance convergence speed, accuracy, and generalization.

4 Algorithm Design

The core algorithm of the program is a feedforward Artificial Neural Network (ANN), trained with backpropagation and mini-batch Gradient Descent, implemented for binary classification. This project consists of two distinct implementations of this algorithm. While both versions implement identical mathematics, the difference lies in their representation. The `base_python` version uses plain Python with nested lists and manual loops; the `numpy` version uses matrix operations and utilizes vectorisation for computational efficiency.

The algorithm has two phases that run once per mini-batch:

- A **forward pass** that produces a prediction.
- A **backward pass** that uses the error to compute how much each weight contributed to it and adjusts them.

4.1 Layer Initialisation

The architecture of the ANN requires initialisation that considers every node of every layer separately, which we achieve by building each layer individually using the `ANN_layer()` class.

Each layer is defined by the number of incoming connections (`n_neurons_input`), its own number of neurons (`n_neurons_output`), and an `activation_function`. First, weights and biases must be initialised for each neuron. The project uses **He initialisation** for ReLU activations and **Xavier/Glorot initialisation** for sigmoid, as these are the recommended choices for maintaining stable gradient magnitudes at the start of training [7,8].

Activation functions are stored in a nested dictionary mapping, which pairs each activation function by name, the primary function `func` for forward pass and the derivative `deriv` for backward pass.

Additionally, each layer contains allocated storage for intermediate values (`a_s` and `z_s`), as well as error signals (`delta`) and `activation_derivatives` used during backpropagation.

```
1 FUNCTION initialise_weights_and_biases(activation_function, n_input,
2   n_output):
3   # Select scale based on activation function type
4   IF activation_function IN {relu, leaky_relu}:
5       std <- sqrt(2 / n_input)           # He initialisation
6   ELSE:
7       std <- sqrt(2 / (n_input + n_output)) # Xavier initialisation
8   # Sample weights and initialise biases to zero
9   weights <- random_normal(mean=0, std=std, shape=(n_output, n_input))
10  biases <- zeros(shape=(n_output, 1))
11
12  RETURN weights, biases
```

Listing 1: Weight and Bias Initialisation

In the **base Python version**, this is realised using `random.gauss()` applied to each individual neuron in the weight matrix through nested loops within a list of lists. Biases are initialised to zero using the same iterative approach.

In the **NumPy version**, the `rng.normal()` method produces the entire weight matrix in a single call. Biases are efficiently set to zero using the `np.zeros()` function, creating a column vector of the required dimensions.

4.2 Forward Pass

4.2.1 Single Sample

The forward pass computes the output of a layer by applying the affine transformation, followed by the activation function. Both pre-activation and post-activation values are stored within the layer, as they are required for training with backpropagation.

```
1 FUNCTION forward(input_vector, weights, biases, activation_function):
2     # Affine transformation (linear step)
3     z <- weights @ input_vector + biases
4
5     # Non-linear activation step
6     a <- activation_function(z)
7
8     # Store intermediate values for backpropagation
9     STORE z, a
10    RETURN a
```

Listing 2: Layer Forward Pass

In the **base Python version**, the matrix multiplication is explicitly written as nested loops in the `_matrix_multiply` method, and bias addition is performed element-by-element in `_add_biases`. The activation function is applied to each element in a similar manner, by looping through the list.

In the **NumPy version**, this computation is condensed into the operation `self.weights @ input_vector + self.biases`. This handles the entire calculation using NumPy's operator overloading. Subsequently, a vectorized lambda operation `self.a_s = func(self.z_s)` applies the activation function to all elements simultaneously.

The network-level forward pass functions by chaining together all layer calls sequentially:

```
1 FUNCTION predict(input_vector, layers):
2     x <- input_vector
3     FOR EACH layer IN layers:
4         x <- layer.forward(x)
5     RETURN x
```

Listing 3: Network Prediction Loop

4.2.2 Batch Processing

The NumPy version processes an entire batch in a single pass by representing it as a matrix of shape `(n_features, batch_size)` rather than iterating over individual samples. The standard `prediction()` method already accepts any 2D input of this shape (a single sample being simply the case `batch_size = 1`). The base-Python version, in contrast, achieves batch processing by calling its single-sample forward pass repeatedly inside a loop.

4.3 Loss Functions

After the forward pass, the ANN's prediction is compared to the true label using a loss function. The first available loss function is the basic Mean Squared Error (MSE), and the second is Binary Cross-Entropy (BCE). BCE is the standard choice for binary classification and is the primary function utilized in this project.

```
1 FUNCTION binary_cross_entropy(y_true, y_pred):
2     # Prevent numerical instability (log of zero)
3     y_pred_safe <- clip(y_pred, 1e-15, 1 - 1e-15)
4
5     # Calculate the log loss for both classes
6     loss_sum <- y_true * log(y_pred_safe) + (1 - y_true) * log(1 -
7     y_pred_safe)
8
9     RETURN -mean(loss_sum)
```

Listing 4: Binary Cross-Entropy Loss Calculation

The derivative is then used during backpropagation to adjust the weights.

```
1 FUNCTION binary_cross_entropy_deriv(y_true, y_pred):
2     # Prevent numerical instability for division
3     y_pred_safe <- clip(y_pred, 1e-15, 1 - 1e-15)
4
5     # Calculate the gradient w.r.t. predictions
6     # Derivative of -[y*log(p) + (1-y)*log(1-p)]
7     gradient <- (y_pred_safe - y_true) / (y_pred_safe * (1 - y_pred_safe))
8
9     RETURN gradient
```

Listing 5: Binary Cross-Entropy Gradient

In both versions, loss functions are implemented using the same technique as for activation functions: a nested `LOSS_FUNCTIONS` dictionary mapping the function's name to its formula and derivative, which are both stored as lambdas.

4.4 Backpropagation: Error Delta Computation

Backpropagation computes the gradient of the loss with respect to every weight and bias in the network by applying the chain rule layer by layer, moving from the output back toward the input. The error signal at each layer is represented by **delta** (δ), which signifies how much the loss changes with respect to the pre-activation value **z** of each neuron.

```
1 FUNCTION compute_deltas(y_batch):
2     # Align labels with activation matrix shape (n_outputs, batch_size)
3     y_true <- transpose(y_batch)
4
5     # 1. Output Layer Delta
6     output_layer <- layers[last]
7     output_layer.compute_activation_derivatives()
8
9     # delta = dL/da * f'(z)
10    loss_deriv <- calculate_loss_deriv(y_true, output_layer.activations)
```

```

11 output_layer.delta <- loss_deriv * output_layer.activation_derivatives
12
13 # 2. Hidden Layers Backwards Loop
14 # Move from the last hidden layer back to the first layer
15 FOR i FROM (number_of_layers - 2) DOWN TO 0:
16     layer <- layers[i]
17     next_layer <- layers[i + 1]
18
19     layer.compute_activation_derivatives()
20
21     # Propagate error: (W_next^T @ delta_next) * f'(z_curr)
22     weighted_error <- transpose(next_layer.weights) @ next_layer.delta
23     layer.delta <- weighted_error * layer.activation_derivatives
24
25 RETURN

```

Listing 6: Backward Pass: compute_deltas()

The activation derivatives $f'(z)$ are computed by layer's `compute_activation_derivatives` method, which evaluates the derivative of the stored activation function at the stored pre-activation values z . For ReLU, this is simply 1 where $z > 0$ and 0 elsewhere. For sigmoid, it is calculated as $\sigma(z) \times (1 - \sigma(z))$.

In the **base Python version**, the weighted sum is computed by a Python loop over neurons and layers, with deltas stored as plain lists. In the **NumPy version**, the operation `np.dot(next_layer.weights.T, next_layer.delta)` computes the weighted sum for the entire batch simultaneously, representing deltas as 2D arrays of shape `(n_neurons, batch_size)`.

4.5 Gradient Computation

The backpropagation algorithm calculates the delta values, from which we can directly get the gradients for each layer's weights and biases, using the chain rule.

```

1 FUNCTION compute_gradients(layers, input_vector):
2
3     FOR i, layer IN enumerate(layers):
4         IF i == 0:
5             prev_a <- input_vector
6         ELSE:
7             prev_a <- layers[i - 1].activations
8
9         # Weight gradient: outer product
10        dweights <- layer.delta @ transpose(prev_a)
11
12        # Bias gradient: delta is the bias gradient
13        dbiases <- layer.delta

```

Listing 7: Gradient Computation

The weight gradient is the outer product of the delta vector with the previous layer's activations. That captures the impact of a given weight on the loss. The bias gradient is the delta itself.

In mini-batch mode, the gradients are then averaged across all samples in the batch, to provide a stable estimate of the true gradient. In the **NumPy batch version**, `dweights` is divided

by the batch size to get the average gradient, and `dbiases` is averaged using `mean(axis=1)`. The **base Python version** stores all gradients across samples and makes the divisions at the end.

4.6 Parameter Update with L2 Regularisation

After the gradients are computed, each layer's weights and biases are updated following the gradient descent. The learning rate is a hyperparameter and it optionally decays over time. That ensures large initial changes, followed by finer convergence at the end. Optionally, L2 regularisation can be used to add a penalty proportionate to the magnitude of the weights (essentially penalizing larger weights), which discourages overfitting by keeping weights small. Biases are updated without regularisation.

```
1 FUNCTION update_parameters(layer, learning_rate, l2_lambda):
2     # L2 regularisation adds derivative of penalty term (lambda * w)
3     # to weight gradient
4     layer.weights <- layer.weights - learning_rate * (dweights + l2_lambda *
5     layer.weights)
6     layer.biases <- layer.biases - learning_rate * dbiases
7     CLEAR dweights, dbiases, delta, z, a # free intermediate state
```

Listing 8: Parameter Update with L2 Regularisation

Both versions implement this in the `update_parameters` method. After updating, all intermediate values (activations, pre-activations, deltas and gradients) are set to `None` or an empty list, to ensure that the next batch starts in a clean state.

4.7 Training Strategy

In the core of training strategy is the gradient-descent loop through epochs and their batches. To improve generalisation and handling of the dataset, several additional techniques are applied. Oversampling is handled in `main` before `train()` is called; early stopping, learning-rate decay, and the mini-batch loop are all integrated inside `train()`.

4.7.1 Class Imbalance — Oversampling

The training data is imbalanced: the two binary classes are not equally represented. When a network trains on imbalanced data, gradient updates are dominated by the majority class, which causes the algorithm to predict the majority class for every input, since that gives a low enough loss. To counteract this, the minority class is oversampled before using the data for training.

```
1 FUNCTION oversample_minority(X_train, Y_train):
2     ones_idx <- indices where label == 1
3     zeros_idx <- indices where label == 0
4
5     diff <- |ones_idx| - |zeros_idx|
6
7     IF diff > 0: # class 1 is majority
```

```
8     extra_idx <- random_sample(zeros_idx, k=diff,
9                               with_replacement=True)
10    X_train <- X_train + [X_train[i] for i in extra_idx]
11    Y_train <- Y_train + [Y_train[i] for i in extra_idx]
12
13    shuffle(X_train, Y_train)
14    RETURN X_train, Y_train
```

Listing 9: Class Imbalance - Oversampling

Duplicates are drawn randomly with replacement from the minority class until both classes are equally represented. The combined dataset is then shuffled so that duplicates are not clustered in mini-batches. Oversampling is applied only to the training set, with the validation set being left untouched so that validation loss reflects the real-world class distribution.

4.7.2 Training Loop: Mini-Batch Gradient Descent with Early Stopping and Learning-Rate Decay

Training a neural network means finding the weights that minimise the loss function over the entire dataset. The most straightforward approach is computing the gradient over all training samples before taking a single update step: full-batch gradient descent. This is computationally expensive on large datasets, since one gradient step requires a full forward and backward pass over every sample.

Mini-batch gradient descent is the standard practical compromise. The training set is divided into small batches of size B , and one gradient update is performed per batch. The gradient is averaged over the B samples in the batch, giving a less noisy estimate than a single sample while remaining far cheaper than a full-dataset pass. One complete pass through all batches, processing every training sample exactly once, constitutes one epoch. At the start of each epoch the dataset is shuffled so that batches have different compositions each time, preventing the network from fitting to any ordering in the data.

In this implementation, the mini-batch loop is extended with two additional mechanisms: learning-rate decay and early stopping. Both are integrated directly inside `train()` function.

```
1 FUNCTION train(X, Y, X_val, Y_val, epochs, learning_rate,
2               batch_size, lr_decay, decay_every,
3               l2_lambda, patience):
4
5     current_lr      <- learning_rate
6     best_val_loss   <- infinity
7     best_weights    <- None
8     epochs_no_improve <- 0
9
10    FOR epoch IN 1..epochs:
11
12        # Learning rate decay
13        IF epoch > 1 AND (epoch - 1) % decay_every == 0:
14            current_lr <- current_lr * lr_decay
15
16        # Shuffle dataset at the start of each epoch
17        shuffle(X, Y)
18
```

```
19 # Mini-batch gradient descent
20 FOR each mini-batch (batch_X, batch_Y) of size batch_size:
21
22     compute_gradients_batch(batch_X, batch_Y)
23     # gradients are averaged over the B samples in the batch
24
25     FOR each layer:
26         update_parameters(current_lr, l2_lambda)
27
28 # Early stopping: evaluate on validation set after each epoch
29 val_loss <- compute_loss(X_val, Y_val)
30
31 IF val_loss < best_val_loss:
32     best_val_loss <- val_loss
33     best_weights <- save_parameters_snapshot()
34     epochs_no_improve <- 0
35 ELSE:
36     epochs_no_improve <- epochs_no_improve + 1
37
38 IF epochs_no_improve >= patience:
39     BREAK
40
41 restore_parameters_snapshot(best_weights)
```

Listing 10: Training Strategy

Learning-rate decay is important, because a learning rate large enough to make fast initial progress tends to overshoot the loss minimum in later epochs, when the model is already close to convergence. Decay resolves this by multiplying the learning rate by a constant factor every fixed number of epochs, producing an exponential decay schedule that allows large steps early, followed by increasingly fine-grained updates as training progresses. The decay factor and interval are passed as parameters, keeping the schedule configurable without changing the algorithm.

Early stopping monitors validation loss after every epoch, and halts training when it has not improved for `patience`-number of consecutive epochs. This guards against overfitting; without it, a network trained for too many epochs will continue to minimise training loss while validation loss rises, meaning it is memorising training samples rather than learning patterns in the data. When training stops, the weights are not taken from the final epoch but restored from the snapshot saved at the epoch with the lowest validation loss, so the returned model is the best-generalising version seen during the run rather than simply the last one. Weight snapshots are implemented by deep-copying all weight and bias matrices, since Python list assignment and NumPy array assignment are both just adding another reference to the object - a shallow copy like that would silently update the snapshot on subsequent steps.

In the **base Python version**, shuffling is done by building a list of indices and constructing new lists via comprehension. In the **NumPy version**, `np.random.permutation` generates a permuted index array and fancy indexing reorders the data arrays in one operation. Both produce the same result; the NumPy version avoids the Python loop over indices.

4.8 Evaluation

The evaluation logic is mathematically identical in both versions. The difference is entirely in how predictions are obtained and how comparisons are made.

In the **base Python version**, `accuracy`, `confusion_matrix`, and `classification_metrics` each iterate over samples one by one in a Python loop, calling `ann.prediction(x)` per sample and comparing scalar values.

In the **NumPy version**, a helper `_to_batch` first normalises the input arrays to consistent shapes — `X` to $(n_features, N)$ and `Y` to $(1, N)$ — handling transposition automatically regardless of how the caller passed them in. `ann.prediction(X)` is then called once for the entire dataset, returning a $(1, N)$ array of probabilities. All comparisons are vectorised with NumPy boolean operations rather than per-sample conditionals.

4.8.1 Accuracy

```
1 FUNCTION accuracy(ann, X, Y, threshold=0.5):
2
3     # Base Python
4     correct <- 0
5     FOR each (x, y) IN (X, Y):
6         pred <- 1 IF ann.prediction(x) >= threshold ELSE 0
7         IF pred == y: correct <- correct + 1
8     RETURN correct / |X|
9
10    # NumPy
11    X, Y <- normalise_shapes(X, Y)      # (n_features, N), (1, N)
12    preds <- ann.prediction(X)          # (1, N) full batch in one call
13    labels <- (preds >= threshold).astype(int)
14    RETURN mean(labels == Y)            # vectorised comparison
```

Listing 11: Accuracy of the model

Each sample is classified by thresholding the network’s raw output probability. The default threshold of 0.5 assigns the positive class when the predicted probability exceeds 50%.

4.8.2 Confusion Matrix

When data is imbalanced, accuracy alone is a misleading parameter. The confusion matrix decomposes predictions into four counts that expose where errors are being made.

```
1 FUNCTION confusion_matrix(ann, X, Y, threshold=0.5):
2
3     # Base Python
4     tn <- fp <- fn <- tp <- 0
5     FOR each (x, y) IN (X, Y):
6         pred <- 1 IF ann.prediction(x) >= threshold ELSE 0
7         label <- y
8         IF label == 0 AND pred == 0: tn <- tn + 1 # correct rejection
9         ELIF label == 0 AND pred == 1: fp <- fp + 1 # false alarm
10        ELIF label == 1 AND pred == 0: fn <- fn + 1 # missed detection
11        ELIF label == 1 AND pred == 1: tp <- tp + 1 # correct detection
```

```
12 RETURN {tn, fp, fn, tp}
13
14 # NumPy
15 X, Y <- normalise_shapes(X, Y)
16 preds <- (ann.prediction(X) >= threshold).astype(int).flatten()
17 labels <- Y.flatten()
18 tn <- sum((labels == 0) AND (preds == 0)) # boolean mask, no loop
19 fp <- sum((labels == 0) AND (preds == 1))
20 fn <- sum((labels == 1) AND (preds == 0))
21 tp <- sum((labels == 1) AND (preds == 1))
22 RETURN {tn, fp, fn, tp}
```

Listing 12: Confusion Matrix

4.8.3 Classification Metrics

From the confusion matrix, four additional metrics are derived that together give a complete picture of classifier behaviour. The formula is identical in both versions; the only difference is that the base Python version calls a loop-based `confusion_matrix` while the NumPy version calls a vectorised one. The metric arithmetic itself operates on four scalar counts in both cases.

```
1 FUNCTION classification_metrics(ann, X, Y, threshold=0.5):
2   {tn, fp, fn, tp} <- confusion_matrix(ann, X, Y, threshold)
3
4   accuracy <- (tp + tn) / (tp + tn + fp + fn)
5   precision <- tp / (tp + fp)
6   recall <- tp / (tp + fn)
7   specificity <- tn / (tn + fp)
8   f1 <- 2 * precision * recall / (precision + recall)
9
10  RETURN {accuracy, precision, recall, specificity, f1}
```

Listing 13: Additional Metrics to evaluate model performance

Precision and recall trade off against each other depending on the decision threshold. F1 is their harmonic mean, giving a single score that penalises imbalance between the two — it is low if either is low. Specificity is the recall analogue for the negative class. Each metric guards against a different failure mode: a model that predicts all positives has perfect recall but zero precision, a model that predicts all negatives has perfect specificity but zero recall. Reporting all four together makes these degenerate cases visible.

4.8.4 Evaluate and Report

`evaluate()` assembles loss, all classification metrics, and the confusion matrix into a single printed report for one dataset. `report_results()` is a lighter version that prints only training and validation accuracy side by side, intended for quick checks during development rather than final evaluation.

```
1 FUNCTION evaluate(ann, X, Y, threshold, name):
2   [NumPy only] X, Y <- normalise_shapes(X, Y)
3   loss <- ann.compute_loss(X, Y)
4   metrics <- classification_metrics(ann, X, Y, threshold)
```

```
5 PRINT name, loss, accuracy, precision, recall, specificity, f1
6 PRINT confusion_matrix(ann, X, Y, threshold)
7 RETURN metrics + {loss}
```

Listing 14: Evaluate and Report

The NumPy version additionally calls `_to_batch` at the top of `evaluate()` to normalise shapes before passing data to `compute_loss`, since the NumPy ANN expects $(n_features, N)$ rather than a list of column vectors.

4.9 Data Parsing

Input data is stored in `.howlin` files, where each line encodes one sample: 27 whitespace-separated feature values followed by a single binary label. The parsing module converts these into the format expected by the ANN.

The low-level functions `parse_line` and `iterate_input` are identical across both versions. The divergence happens in `parse_input`, where the base Python version reshapes each sample into a column vector immediately, while the NumPy version accumulates flat rows and converts them in bulk.

```
1 FUNCTION parse_line(line):
2     parts <- line.strip().split()
3     IF |parts| != 28: RAISE ValueError
4     values <- [float(x) for x in parts[0:27]]
5     label <- [int(parts[27])]
6     RETURN values, label
7
8 FUNCTION iterate_input(filename, start=None, end=None):
9     FOR i, line IN enumerate(open(filename)):
10         IF i < start: CONTINUE
11         IF i >= end: BREAK
12         YIELD parse_line(line)
13
14 FUNCTION parse_input(filename, start=None, end=None):
15
16     # Base Python
17     input_list <- []
18     label_list <- []
19     FOR values, label IN iterate_input(filename, start, end):
20         input_list.append([v for v in values]) # column vector (27, 1)
21         label_list.append([v for v in label])  # column vector (1, 1)
22     RETURN input_list, label_list              # lists of column vectors
23
24     # NumPy
25     input_list <- []
26     label_list <- []
27     FOR values, label IN iterate_input(filename, start, end):
28         input_list.append(values) # flat list, no reshaping yet
29         label_list.append(label)
30     X <- np.array(input_list, dtype=float32) # shape (N, 27)
31     y <- np.array(label_list, dtype=int32)  # shape (N, 1)
32     RETURN X, y
```

Listing 15: Input Data

`iterate_input` is a generator that streams the file line by line, to avoid loading the entire dataset into memory at once. The `start` and `end` parameters allow loading a contiguous subset of lines, which can be used to separate training, validation, and test splits.

The base Python version reshapes each sample into a (27, 1) column vector immediately, matching the format the base ANN expects for individual `prediction()` calls. The NumPy version accumulates flat rows and converts them in a single `np.array()` call at the end, producing an (N , 27) matrix. The NumPy ANN then transposes this to (27, N) internally when needed for batched operations.

`load_and_split` exists only in the base Python version as an alternative generator-based entry point for stochastic training, yielding shuffled mini-batches directly from the file. The NumPy version does not need this because its `train()` method handles shuffling and batching internally via `np.random.permutation` and array slicing.

```
1 FUNCTION load_and_split(filename, batch_size):
2     inputs, labels <- parse_input(filename)
3     data <- zip(inputs, labels)
4     shuffle(data)
5     FOR i IN 0, batch_size, 2*batch_size, ...:
6         batch <- data[i : i + batch_size]
7         YIELD unzip(batch) as (input_batch, label_batch)
```

Listing 16: Load and Split Data

4.10 Summary of Base Python vs NumPy Differences

The two implementations share identical algorithms and pseudocode. All differences are in how those algorithms are expressed in the actual code: the base Python version uses nested lists and explicit loops, making each step visible, while the NumPy version replaces those loops with vectorised matrix operations.

Aspect	Base Python	NumPy
Matrix multiply	Explicit loop in <code>weights @ input_vector</code> <code>_matrix_multiply</code>	
Activation function	Element-wise list comprehension	Vectorised lambda on arrays
Delta (single sample)	List of scalars	2D array $(n, 1)$
Delta (batch)	Loop over samples	2D array $(n, batch_size)$
Gradient averaging	Accumulator loop, divide at end	<code>/ batch_size</code> , <code>mean(axis=1)</code>
Weight update	Nested list comprehension	In-place NumPy arithmetic
Batch forward pass	Loop over samples calling <code>prediction()</code>	Single <code>_forward_batch()</code> matrix call
Prediction input shape	List of $(n, 1)$ column vectors	2D array $(n_features, N)$
Evaluation loop	Per-sample Python loop	Vectorised boolean masks on full arrays
<code>parse_input</code> output	List of $(27, 1)$ column vectors	$(N, 27)$ float32 array
Batch shuffling	Index list + list comprehension	<code>np.random.permutation</code> + fancy indexing
<code>load_and_split</code>	Generator yielding mini-batches	Not needed — handled inside <code>train()</code>
Shape normalisation in evaluation	Not needed — consistent list format	<code>_to_batch()</code> aligns X/Y before evaluation

Table 1: Summary of implementation differences between the base Python and NumPy versions.

5 Program Design

The program is organised into two parallel implementations, `base_python_version` and `numpy_version`. They share the same file structure and overall design. The base Python version uses only the Python standard library, while the NumPy version replaces manual matrix operations with NumPy arrays. The description below is mainly based in the non-NumPy version, but all possible structural differences between versions are noted.

5.1 Repository Structure

```
ANN_Backprop_Implement/
├── src/
│   ├── base_python_version/
│   │   ├── ANN_layer_base_python.py ..... Layer class
│   │   ├── ANN_base_python.py ..... Network class
│   │   ├── data_input_base_python.py ..... Input parsing
│   │   └── evaluate_base_python.py ..... Evaluation utilities
│   └── numpy_version/
│       ├── ANN_layer_numpy.py
│       ├── ANN_numpy.py
│       ├── data_input_numpy.py
│       └── evaluate_numpy.py
├── results/
│   ├── main_base_python.py ..... Entry point (base Python)
│   └── main_numpy.py ..... Entry point (NumPy)
├── data/ ..... .howlin files for training and testing
├── models/ ..... Saved model weights and parameters
└── test/ ..... Pytest test suites
```

5.2 Files and Responsibilities

`data_input_base_python.py` is responsible for input parsing. It reads the raw `.howlin` data files from the `data/` directory and converts each line into a pair of vectors: an input vector $\mathbf{x}$ of 27 features, and a label vector $\mathbf{y}$. This file has no connection to the network and only handles data parsing in a format that the rest of the program expects. It has the handy option to parse only parts of the file using an iterator, however, this option is not being used in the final implementation.

`ANN_layer_base_python.py` defines the `ANN_Layer` class, which represents a single layer of the network. It owns the weights matrix and bias vector for that layer, and contains all per-layer operations: the forward pass, activation derivative computation, and parameter updates. A layer object is in this way tidy and independent, only aware of its own inputs and outputs.

`ANN_base_python.py` defines the `ANN` class, which owns a list of `ANN_Layer` objects and builds the entire network with all required methods. It is responsible for building the lay-

ers and running the entire forward pass by calling the forward pass of each separate layer. Then it runs backpropagation across all layers in reverse order, and executes the training loop using the `train()` function. Since it builds the entire model, it also has the methods to save the trained model's information and parameters into a text file and to load it back for inference.

`evaluate_base_python.py` receives a trained ANN object and a dataset, runs predictions, and computes performance metrics including accuracy, precision, recall, F1-score, and a confusion matrix. It does not run backpropagation or modify the network in any way. It only provides results about the model's performance.

`results/main_base_python.py` is, as name tells, the main file. It runs the entire pipeline of the program and it is the only file that needs to be executed directly. It imports all of the modules mentioned above, and defines the network architecture and training hyperparameters, which the user can provide and change.

`test/` contains pytest test suites organised by implementation (`test_base_python/` and `test_numpy/`). Each test file targets a specific method or component, such as `test_forward_pass.py` and `test_update_parameters.py`, and verifies correctness independently of the rest of the program.

5.3 Program Flow

The pseudocode below shows how control and data move between files during a full training and evaluation run.

For training:

```
1 1. PARSE INPUT DATA
2   data_input_base_python -> parse_input("data/training_set.howlin")
3   returns X_all, Y_all as lists of column vectors
4 2. PREPARE TRAINING DATA
5   main -> shuffle and split X_all, Y_all into:
6           X_train, Y_train (TRAIN_RATIO of the data)
7           X_val, Y_val (remainder)
8   main -> oversample minority class in X_train, Y_train
9   returns X_train, Y_train, X_val, Y_val
```

Listing 17: Training pipeline - input parsing and data preparation

```
1 3. BUILD NETWORK
2   main -> ANN_base_python(n_layers, n_neurons_each_layer,
3                           activation_hidden, activation_output,
4                           loss_function, rng)
5   ANN -> _build_ANN()
6   for each layer i:
7       create ANN_Layer(i, n_input, n_output, activation)
8       call layer.initialize_weights_bias(rng)
```

Listing 18: Training pipeline - network construction

```
1 4. TRAIN NETWORK
2     main -> ann.train(X_train, Y_train, X_val, Y_val,
3                     epochs, learning_rate, batch_size, ...)
4     for each epoch:
5         apply learning rate decay if scheduled
6         shuffle training data
7         for each mini-batch:
8             ANN -> compute_gradients_batch(batch_inputs,
9             batch_targets)
10                for each sample in batch:
11                    ANN -> prediction(x)                # forward pass
12                    ANN -> _compute_deltas(y)            # backward pass
13                    accumulate per-sample gradients
14                    average gradients across batch
15                for each layer:
16                    layer.update_parameters(learning_rate, l2_lambda)
17            ANN -> compute_loss(X_train, Y_train)
18            ANN -> compute_loss(X_val, Y_val)
19            if validation loss improved:
20                ANN -> _save_parameters_snapshot()
21            else if patience exceeded:
22                ANN -> _restore_parameters_snapshot()
23                stop training early
```

Listing 19: Training pipeline - training loop with mini-batch gradient descent and early stopping

```
1 5. REPORT AND SAVE
2     main -> report_results(ann, X_train, Y_train, X_val, Y_val)
3     main -> ann.save_model(model_name, data_name, path="models/")
4     ANN -> writes weights and biases of all layers to models/*.txt
```

Listing 20: Training pipeline - reporting and model persistence

For testing:

```
1 1. LOAD MODEL
2     main -> ANN_base_python.load_model(MODEL_FILE)
3     reads architecture, weights, and biases from models/*.txt
4 2. PARSE TEST DATA
5     data_input_base_python -> parse_input("data/...")
6     returns X_test, Y_test
7 3. EVALUATE
8     main -> evaluate(ann, X_test, Y_test, name=...)
9     evaluate_base_python -> for each sample:
10        ann.prediction(x)
11    evaluate_base_python -> compute accuracy, confusion matrix,
12        precision, recall, F1
13    evaluate_base_python -> print results to stdout
```

Listing 21: Testing pipeline - model loading, data parsing, and evaluation

6 Program Manual

This manual describes how to use the ANN library provided in this project. The codebase exposes two interchangeable implementations, a base-Python version and a NumPy version, that share the same public interface. Two entry-point scripts, `main_base_python.py` and `main_numpy.py`, are provided as worked examples that train and evaluate a network on the dataset supplied with the assignment. They are not part of the core library: their role is to demonstrate the typical workflow and to serve as templates that the user can adapt for their own data and configurations.

6.1 Scope and limitations

The ANN class supports any feedforward architecture with configurable depth, width, and per-layer activation. Within that family, the library is suitable for:

- **Binary classification**, using `sigmoid` output activation with `binary_crossentropy` loss.
- **Scalar or multi-output regression**, using a linear-equivalent output (e.g. `relu`) with `mse` loss.

The library does *not* currently support multi-class classification (no `softmax` activation or categorical cross-entropy loss is implemented), nor does it provide convolutional, recurrent, or other non-dense layer types. Inputs must be feature vectors; non-tabular data such as images or sequences would have to be flattened or otherwise pre-processed before being passed to the network.

6.2 Running the example scripts

Each entry-point script exposes the library through two distinct modes, selected by a single command-line argument:

```
python main_base_python.py train
python main_base_python.py test
```

The two modes correspond to the two halves of a typical machine-learning workflow and are intentionally separated so that they can be run independently:

- **train mode** fits a new model. It loads the training dataset, splits it into training and validation subsets, builds an ANN with the configured architecture, runs the training loop, and saves the resulting model to disk.
- **test mode** evaluates a previously trained model. It loads a saved model from disk, parses a separate test dataset, and reports performance metrics on it. No training happens here, the model's parameters are read from the file and used directly for prediction.

This separation means a user can train a model once and then evaluate it on as many different test sets as they like, without ever re-running the (expensive) training loop.

All script-level settings, file paths, architecture, hyperparameters, and the path to the model being tested, are exposed as constants at the top of the entry-point script and can be edited directly.

6.3 Input format

The network expects data in **column-vector form**. Every sample is represented as a two-dimensional structure with one column and as many rows as there are features. A sample with n_{features} inputs therefore has shape $(n_{\text{features}}, 1)$: the first dimension lists the features, and the second dimension is just a single column holding their values.

The two implementations realise this format with their respective native data types:

- In the **base-Python** version, a sample is a list of single-element lists, for example `[[0.5], [0.2], [0.9]]` for three features.
- In the **NumPy** version, the same sample is a 2-D `numpy.ndarray` of shape `(3, 1)`, constructed from the same nested-list literal.

A **batch** of samples is just several of these column vectors grouped together. The two implementations differ slightly in how they group them:

- The **NumPy** version stacks the columns side by side into a single matrix of shape $(n_{\text{features}}, n_{\text{samples}})$, so each column is one sample. This allows the entire batch to be processed in a single vectorised operation.
- The **base-Python** version keeps the samples as a Python list of individual column vectors, since it lacks the matrix machinery to operate on a stacked batch directly.

Targets follow the exact same convention, with shape $(n_{\text{output}}, n_{\text{samples}})$ – one column per sample, one row per output neuron.

6.4 Train mode

In **train** mode, the entry-point script performs the following steps:

1. Loads and parses the training dataset using `parse_input()`.
2. Shuffles the data and splits it into a training set and a validation set, in the proportion set by `TRAIN_RATIO`.
3. Oversamples the minority class in the training set to address class imbalance (this step is binary-classification-specific).
4. Builds an ANN with the architecture and activation/loss configuration specified at the top of the script.

5. Calls `ann.train(...)` to run the training loop with the configured hyperparameters.
6. Reports training and validation performance with `report_results()`.
7. Saves the trained model to the `models/` directory using `ann.save_model()`.

A network is created by instantiating the ANN class with the desired architecture and activation/loss configuration:

```
1 ann = ANN(  
2     n_layers=3,  
3     n_neurons_each_layer=[n_input, 16, n_output],  
4     activation_hidden="relu",  
5     activation_output="sigmoid",  
6     loss_function="binarycrossentropy",  
7 )
```

Listing 22: Initialization example of ANN

The constructor validates the configuration and rejects invalid combinations (for example, `binarycrossentropy` loss requires a `sigmoid` output activation).

Training is then performed by calling `train()`, passing the training and validation sets along with the desired hyperparameters:

```
1 history = ann.train(  
2     X_train, Y_train, X_val, Y_val,  
3     epochs=200,  
4     learning_rate=0.01,  
5     batch_size=32,  
6     lr_decay=0.95,  
7     decay_every=20,  
8     l2_lambda=1e-4,  
9     patience=50,  
10    verbose=True,  
11 )
```

Listing 23: Training example of ANN

The method returns a `history` dictionary containing the per-epoch training and validation losses, suitable for plotting.

6.5 Test mode

In **test** mode, the entry-point script performs only three steps:

1. Loads a previously saved model with `ANN.load_model(MODEL_FILE)`, reconstructing the trained network from its text file.
2. Parses the test dataset with `parse_input()`.
3. Calls `evaluate()` to compute performance metrics on the test set.

To evaluate a different model or a different test set, only the `MODEL_FILE` and `TEST_DATA_FILE` constants at the top of the script need to be changed; no other code modifications are required.

6.6 Context-dependent components

Two pieces of the example workflow are tightly bound to the specific task addressed in this assignment, and would need to be replaced when applying the library to a different problem:

- `parse_input()` is the dataset loader. It reads the assignment-specific `.howlin` file format and returns `X` and `Y` already arranged in column-vector form. A user with a different dataset must provide their own loader that produces the same output format; once the data is in column-vector form, every other component of the library works without modification.
- `evaluate()` and `report_results()` compute binary-classification metrics (accuracy, confusion matrix, precision, recall, F1). For a regression task, or any task with a different notion of "performance", the user must provide their own evaluation logic. The trained model itself is loaded and queried in exactly the same way regardless.

The core library in `src/` – the `ANN` class, layer class, training loop, save/load logic – is fully task-agnostic and does not need to be modified.

6.7 Configuration options

Network behaviour is controlled entirely through the constructor and `train()` arguments; no internal code needs to be modified to explore different configurations.

Architecture (constructor):

- `n_layers`, `n_neurons_each_layer` – depth and width of the network.
- `activation_hidden`, `activation_output` – activations for the hidden and output layers. Supported: `relu`, `leaky_relu`, `sigmoid`.
- `loss_function` – objective minimised during training. Supported: `mse`, `binarycrossentropy` (the latter requires `sigmoid` output).

Training (`train()` arguments):

- `epochs` – maximum number of training epochs.
- `learning_rate` – initial learning rate used by gradient descent.
- `batch_size` – size of each mini-batch.
- `lr_decay`, `decay_every` – multiplicative learning-rate decay factor and its application interval (in epochs).
- `l2_lambda` – strength of the L2 regularisation term applied to the weights.
- `patience` – number of epochs without validation-loss improvement after which early stopping is triggered.
- `verbose` – whether to print per-epoch progress to the console.

6.8 Saving and loading models

After training, a network can be persisted with `save_model()`, which writes a human-readable text file containing metadata (training hyperparameters, dataset name, architecture) followed by the weight matrices and bias vectors of each layer:

```
ann.save_model("my_model.txt", data_name="my_dataset", path="../models")
```

A previously saved model can be reconstructed with the class method `ANN.load_model()`:

```
ann = ANN.load_model("../models/my_model.txt")
```

The returned `ANN` instance has the saved parameters loaded; predictions made by it are numerically identical to those of the original network within the precision of the saved file.

6.9 Expected output

When `verbose=True`, the program prints one progress line per epoch reporting the training and validation losses, along with notifications when the learning rate is decayed or early stopping is triggered. At the end of training, the network's parameters are restored to those of the best-performing epoch (lowest validation loss). A typical run produces output of the form:

```
Epoch 1/200 - Train Loss: 0.682341 | Val Loss: 0.679018
Epoch 2/200 - Train Loss: 0.654277 | Val Loss: 0.651893
...
[LR decayed to 0.009500]
...
*** Early stopping at epoch 137 (best was epoch 87) ***

Restoring best weights from epoch 87 (val loss: 0.214068)
Model saved to: /.../models/trained_model.txt
```

6.10 Adapting the example scripts

To apply the library to a new dataset or task, the user typically only needs to modify the entry-point script:

1. Replace the call to `parse_input()` with a loader appropriate for the new data, ensuring the resulting `X` and `Y` are in column-vector format.
2. Adjust `N_NEURONS_EACH_LAYER` so the first and last entries match the input and output dimensions of the new dataset.
3. Choose `activation_output` and `loss_function` appropriate for the task (sigmoid + BCE for binary classification, ReLU + MSE for regression).
4. Replace the evaluation logic in `test` mode if the task is not binary classification, since `evaluate()` reports binary-classification metrics by default.

5. Tune the training hyperparameters (`epochs`, `learning_rate`, `batch_size`, etc.) as needed.

The core library code in `src/` does not need to be modified for any of these changes.

7 Results

This section presents the results of training and evaluating the ANN on a binary classification task. The purpose is to demonstrate that the implementation is functioning correctly: the network learns, converges, and produces meaningful predictions. We did not spend a significant amount of time for the optimisation of classification performance and the run is relatively illustrative.

7.1 Experimental Setup

Both versions were trained and evaluated on the same dataset under identical hyperparameter settings. The dataset is the first 3 `.howlin` files combined and consists of 4,035 samples with 27 input features and a binary label.

Parameter	Value
Architecture	[27, 32, 16, 1] (4 layers)
Hidden activation	ReLU
Output activation	Sigmoid
Loss function	Binary cross-entropy
Epochs (max)	200
Learning rate	0.01
LR decay factor	0.95 every 20 epochs
Batch size	32
L2 regularisation	$\lambda = 10^{-4}$
Early stopping patience	50
Train / val split	85% / 15% (3,429 / 606 samples)
Test set	1,344 samples (held-out file)
Random seed	42

Table 2: Hyperparameter settings used for both implementations.

The dataset is imbalanced: 62.11% of samples belong to class 1 and 37.89% to class 0. Minority class oversampling was applied to the training set before training, balancing both classes to 2,133–2,138 samples each. The validation and test sets were left untouched.

It should also be noted that the two implementations rely on different random number generators: the base Python version uses the standard random module, while the NumPy version uses `numpy.random`. Although both generators are initialised with the same seed, they produce different sequences and therefore different shuffles, weight initialisations, and oversampling outcomes.

7.2 Training Convergence

Both implementations converged over the course of training, with training and validation loss decreasing consistently across epochs.

Version	Epochs run	Best epoch	Best val loss	Stopped by
Base Python	134	84	0.5217	Early stopping
NumPy	200	196	0.5253	Max epochs

Table 3: Training convergence summary for both implementations.

The base Python model triggered early stopping at epoch 134, having seen no improvement in validation loss since epoch 84. The NumPy model continued to improve slowly through all 200 epochs, with the best weights recorded at epoch 196. Both models exhibit the expected pattern of train and validation loss declining together with modest separation, consistent with learning without severe overfitting.

7.3 Classification Results

After training, both models were evaluated on the held-out test set of 1,344 samples. Parameters of both the base Python and Numpy model were evaluated by both evaluators. The metric values produced by the evaluators were the same, which confirms that implementations are arithmetically equivalent.

Metric	Base Python model	NumPy model
Test loss	0.4934	0.4852
Accuracy	76.41%	77.31%
Precision	81.43%	81.62%
Recall	80.36%	81.92%
Specificity	69.94%	69.74%
F1 score	80.89%	81.77%

Table 4: Test set classification metrics for both trained models. Evaluated on 1,344 held-out samples.

Both models achieve comparable test accuracy (76–77%) and F1 score (81%), which is meaningfully above the majority-class baseline of 62.11%. The similarity in results confirms that both implementations converge to comparable solutions when trained under the same conditions. Minor differences are attributable to the different number of epochs completed and the stochastic nature of mini-batch training, not to any algorithmic divergence between the two versions.

7.3.1 Confusion Matrices

Version	TN	FP	FN	TP
Base Python	356	153	164	671
NumPy	355	154	151	684

Table 5: Confusion matrix counts on the test set (1,344 samples, threshold = 0.5). TN = true negative, FP = false positive, FN = false negative, TP = true positive.

Both models make similar errors. The main asymmetry visible in both is between specificity ($\approx 70\%$) and recall ($\approx 81\%$), meaning the models are better at identifying true positives than true negatives. The original data still retains a small imbalance after oversampling, which could explain this result.

7.4 Empirical Runtime

The Big O analysis established that both implementations share the same asymptotic complexity class. Here we measured the empirical time to understand the change of the constant factor, wall-clock time, between the implementations. The runtimes below were measured on the same hardware under the same conditions as the training runs above.

Operation	Base Python	NumPy
Training (134 / 200 epochs)	332.61 s	9.10 s
Evaluation on test set (1,344 samples)	0.54–0.61 s	0.007–0.011 s
Speedup (training)	$\approx 37\times$	
Speedup (evaluation)	$\approx 60\times$	

Table 6: Measured runtimes. The base Python model ran for 134 epochs (early stopping); the NumPy model ran for 200 epochs.

The base Python model completed 134 epochs in 332.6 seconds, giving approximately **2.48** seconds per epoch. The NumPy model completed 200 epochs in 9.10 seconds, giving approximately **0.046** seconds per epoch.

Evaluation of the same 1,344-sample test set took 0.54–0.61 seconds with the base Python evaluator and 0.007–0.011 seconds with the NumPy evaluator, a speedup of approximately $60\times$.

These results are consistent and expected, as the base Python version reaches every multiply-add through at least three nested Python `for` loops, incurring interpreter overhead (bytecode dispatch, dynamic type checking, reference counting) on every iteration [11,12]. The NumPy version replaces the innermost loops with single BLAS calls executed in compiled C, with no Python-level iteration over individual elements [13]. Additionally, NumPy stores arrays in contiguous memory blocks that allow efficient CPU cache utilisation, whereas Python lists store pointers to heap-allocated objects scattered across memory, incurring frequent cache misses [13].

8 Runtime Analysis

Runtime analysis was done simultaneously for both versions of the program, the base Python implementation and the NumPy implementation. Where relevant, the two are compared directly.

Throughout this section the following symbols are used:

- $O(\cdot)$ — Big O notation. It describes how the running time of an algorithm scales with the size of its input. Constants and lower-order terms are dropped: $O(2n + 3) = O(n)$, $O(n^2 + n) = O(n^2)$.
- *out* — `n_neurons_output`: the number of neurons in the *current* layer (i.e. the number of rows in the weight matrix).
- *in* — `n_neurons_input`: the number of neurons in the *previous* layer (i.e. the number of columns in the weight matrix).
- B — batch size: the number of training samples processed simultaneously in one forward/backward pass. For a single sample $B = 1$, which reduces every NumPy batch expression to the same class as the base-Python equivalent.

It is important to note that NumPy operations such as `weights @ input_vector` or `np.dot(A, B)` are **not** any more efficient in terms of big O notation. They perform the same number of arithmetic operations as the equivalent pure-Python loops, but those are executed in optimised C/BLAS routines, so the constant factor is a lot smaller. The asymptotic complexity class is therefore identical to the base-Python version; only the actual time-per-operation constant changes. [13]

8.1 ANN Layer

8.1.1 $O(1)$ Operations

Wherever an operation is an instance check or a variable assignment, the Big O notation is $O(1)$. The entire `__init__` method of the ANN layer therefore equals $O(1)$, both in the base Python and NumPy version, as it consists exclusively of scalar checks and attribute assignments with no loops.

The `shape()` method is $O(1)$ as well, since it only reads the lengths of weights, biases, or output without performing any iteration.

8.1.2 Weight and Bias Initialisation, Matrix Operations

`weights_matrix (setter)`

In **base Python** the setter begins with two validation checks that each iterate over all weights, costing $O(out \times in)$ each. The final assignment to `self.weights` is a reference copy and therefore $O(1)$. Summing: $O(2 \times out \times in + 1) = O(out \times in)$.

In **NumPy**, the list-path validation iterates over every value: $O(out \times in)$. If the input is a list, conversion via `np.array()` also copies every element into a new contiguous buffer, again $O(out \times in)$. Shape checks are $O(1)$. The final assignment is a reference copy, $O(1)$.

Dominant term: $O(out \times in)$.

Operation	Base Python	NumPy
Validation (iterate all values)	$O(out \times in)$	$O(out \times in)$
<code>np.array()</code> conversion	—	$O(out \times in)$
Shape checks	$O(1)$	$O(1)$
Assignment to <code>self.weights</code>	$O(1)$	$O(1)$
Final Big O	$O(out \times in)$	$O(out \times in)$

Table 7: Big O analysis of `weights_matrix` setter.

`biases_vector` (setter)

The bias vector has shape $(out, 1)$, so the same reasoning applies but with a smaller matrix. Both validation passes iterate over out values, giving $O(2 \times out) = O(out)$ for both versions.

Operation	Base Python	NumPy
Validation (iterate all values)	$O(out)$	$O(out)$
<code>np.array()</code> conversion	—	$O(out)$
Assignment to <code>self.biases</code>	$O(1)$	$O(1)$
Final Big O	$O(out)$	$O(out)$

Table 8: Big O analysis of `biases_vector` setter.

`initialize_weights_bias()`

Base Python has a nested list comprehension that fills the weight matrix cell by cell. The outer loop runs out times, the inner loop runs in times, calling `rng.gauss()` once per cell. Bias initialisation fills out values. Dominant term: $O(out \times in)$.

NumPy's `rng.normal(0, std, size=(out, in))` generates and writes $out \times in$ random values in a single C-level call. `np.zeros((out, 1))` initialises out bias values. Dominant term: $O(out \times in)$.

Operation	Base Python	NumPy
Weight matrix fill	$O(out \times in)$	$O(out \times in)$
Bias vector fill	$O(out)$	$O(out)$
Final Big O	$O(out \times in)$	$O(out \times in)$

Table 9: Big O analysis of `initialize_weights_bias()`.

`_print_matrix()` / `print_weights_and_biases()`

`_print_matrix()` iterates over each row and each column, making it $O(\text{rows} \times \text{columns})$. `print_weights_and_biases()` calls it once for weights — $O(\text{out} \times \text{in})$, and once for biases — $O(\text{out} \times 1) = O(\text{out})$. Since $\text{in} \geq 1$, the weights term dominates. In NumPy, `np.round()` creates a rounded copy of every element before printing, giving the same cost.

Operation	Base Python	NumPy
Print weights	$O(\text{out} \times \text{in})$	$O(\text{out} \times \text{in})$
Print biases	$O(\text{out})$	$O(\text{out})$
Final Big O	$O(\text{out} \times \text{in})$	$O(\text{out} \times \text{in})$

Table 10: Big O analysis of `print_weights_and_biases()`.

`_matrix_multiply()`

The outer loop runs once per output neuron, so out times. Inside, `sum()` iterates over all input connections for that row, which is in operations. The total number of multiply-add operations is $\text{out} \times \text{in}$.

In **NumPy** this is replaced by `self.weights @ input_vector`. This single BLAS call that performs the same $\text{out} \times \text{in}$ multiply-adds (or $\text{out} \times \text{in} \times B$ for a batch of size B), but in compiled C rather than interpreted Python.

Operation	Base Python	NumPy
Outer loop (output neurons)	$O(\text{out})$	—
Inner <code>sum()</code> (input weights)	$O(\text{in})$ per row	—
Matrix multiply (<code>W @ x</code>)	$O(\text{out} \times \text{in})$	$O(\text{out} \times \text{in} \times B)$
Final Big O	$O(\text{out} \times \text{in})$	$O(\text{out} \times \text{in} \times B)$

Table 11: Big O analysis of `_matrix_multiply()`.

`_add_biases()`

Biases are connected to one neuron layer at a time, so there are out of them. The list comprehension iterates over all elements of the $(\text{out}, 1)$ shaped matrix, performing one addition per output neuron. $O(\text{out} \times 1) = O(\text{out})$. In NumPy the broadcast addition over the (out, B) array is $O(\text{out} \times B)$, which for a single sample reduces to the same $O(\text{out})$.

Operation	Base Python	NumPy
Bias addition	$O(\text{out})$	$O(\text{out} \times B)$
Final Big O	$O(\text{out})$	$O(\text{out} \times B)$

Table 12: Big O analysis of `_add_biases()`.

8.1.3 Forward Pass and Derivatives

`forward()`

The function begins with input validation. Two checks iterate over the input vector, giving $O(2 \times in) = O(in)$. All other type checks are $O(1)$ pointer comparisons. The dominant work is then `_matrix_multiply()` at $O(out \times in)$, followed by `_add_biases()` at $O(out)$, and applying the activation function element-wise at $O(out)$. Summing:

$$O(2 \cdot in + out \cdot in + out + 1) = O(out \times in)$$

In **NumPy**, the matrix multiply handles a full batch: `self.weights @ input_vector` with shapes $(out, in) \times (in, B) \rightarrow (out, B)$, costing $O(out \times in \times B)$. Bias broadcast addition and element-wise activation each cost $O(out \times B)$. Dominant term: $O(out \times in \times B)$, which for $B = 1$ reduces to $O(out \times in)$ — identical to base Python.

Operation	Base Python	NumPy
Input validation	$O(in)$	$O(in \times B)$
Matrix multiply	$O(out \times in)$	$O(out \times in \times B)$
Add biases	$O(out)$	$O(out \times B)$
Activation function	$O(out)$	$O(out \times B)$
Final Big O	$O(out \times in)$	$O(out \times in \times B)$

Table 13: Big O analysis of `forward()`.

`compute_activation_derivatives()`

Value checks here are $O(1)$. The function then computes the derivative of the activation function for each output neuron. The computational cost of one derivative is $O(1)$, and one is required per output neuron, making the total $O(out)$.

In **NumPy**, the derivative function is applied element-wise to the entire (out, B) array `z_s` in one C call — visiting every element exactly once: $O(out \times B)$. For $B = 1$ both versions are identical.

Operation	Base Python	NumPy
State checks	$O(1)$	$O(1)$
Derivative per neuron	$O(out)$	$O(out \times B)$
Final Big O	$O(out)$	$O(out \times B)$

Table 14: Big O analysis of `compute_activation_derivatives()`.

`update_parameters()`

All value and state checks are singular comparisons: $O(1)$ as a group. For the actual update, the outer loop iterates over rows (out) and the inner loop over columns (in), rebuilding

the weight matrix cell by cell. Bias update is a separate loop over *out* values. Dominant term: $O(out \times in)$.

In **NumPy**, `self.weights -= learning_rate * (self.dweights + l2_lambda * self.weights)` is a vectorised in-place operation over the (out, in) array – $O(out \times in)$. The optional L2 term adds another $O(out \times in)$ element-wise multiply, but this does not change the asymptotic class. Bias update is $O(out)$. Dominant term: $O(out \times in)$.

Operation	Base Python	NumPy
Checks	$O(1)$	$O(1)$
Weight update	$O(out \times in)$	$O(out \times in)$
Bias update	$O(out)$	$O(out)$
Cleanup (= None)	$O(1)$	$O(1)$
Final Big O	$O(out \times in)$	$O(out \times in)$

Table 15: Big O analysis of `update_parameters()`.

8.1.4 Overall Layer Complexity

Across the entire layer, the largest matrix that must be iterated over is the weight matrix of shape (out, in) , where *out* is the number of neurons in this layer and *in* is the number of neurons in the previous layer. Every non-trivial operation either touches this matrix once ($O(out \times in)$) or operates on a subset of it. The overall Big O of the ANN layer is therefore:

$$O(out \times in)$$

for a single sample, and

$$O(out \times in \times B)$$

when processing a mini-batch of size *B* in the NumPy version.

8.2 ANN

The following section analyses the computational complexity of the main ANN class, covering both the base Python and NumPy implementations. The same notation defined in the layer section applies, with additional variables introduced:

- *L* — number of computational layers (`n_layers` – 1), i.e. the number of layers excluding the input.
- *N* — total number of training samples in the dataset.
- *E* — number of training epochs.
- *B* — mini-batch size: the number of samples processed in one gradient step. N/B batches are processed per epoch.

A structural difference between the two implementations is that the NumPy version introduces `_forward_batch()`, which processes an entire mini-batch in a single matrix multiply rather than looping over samples one by one. This does not reduce the asymptotic Big O class, but it eliminates the Python-level loop over B samples, meaning all B computations are vectorised into one BLAS call and the constant runtime factor is substantially smaller.

8.2.1 $O(1)$ Functions

Loss Functions

In both Mean Squared Error (MSE) and Binary Cross Entropy (BCE), the `sum()` call iterates over n predictions and is $O(N)$ when called on the full output. When used inside `train()` per sample, the cost equals the number of neurons in the last layer of the ANN — $O(out_{last})$. Since this ANN is designed for binary classification, there is exactly one output neuron in the last layer, which makes the sum over the output equal $O(1)$.

In **NumPy**, both `func` and `deriv` are vectorised operations. When called with (out, B) arrays they cost $O(out \times B)$ each. For binary classification with a single output neuron this again reduces to $O(B)$.

`__init__()`

`__init__` handles the assignment of all instance variables and is composed exclusively of scalar checks and assignments. It contains no loops, so each operation costs $O(1)$, making the final Big O for the method $O(1)$ in both versions.

8.2.2 Network-Level Functions

`_build_ANN()`

To build the network, all L layers must be constructed. As established in the layer analysis, the cost per layer is $O(out_i \times in_i)$, dominated by `initialize_weights_bias()` filling the weight matrix. With L layers the total cost is $O(L \times out \times in)$, or $O(L \times W^2)$ in the uniform-width case where all layers have width W . The asymptotic class is identical for both versions.

Operation	Base Python	NumPy
Loop over L layers	$O(L)$	$O(L)$
<code>initialize_weights_bias()</code> per layer	$O(out \times in)$	$O(out \times in)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in)$

Table 16: Big O analysis of `_build_ANN()`.

`prediction()`

The function passes the input through all L layers sequentially, calling `forward()` on each. As established in the layer analysis, one forward call costs $O(out_i \times in_i)$. Summing over all

layers:

$$O\left(\sum_{i=1}^L out_i \times in_i\right) = O(L \times out \times in)$$

The initial `np.array()` conversion and reshape in the NumPy version cost $O(in)$, which is dominated by the layer passes. Both versions are therefore $O(L \times out \times in)$.

Operation	Base Python	NumPy
Array conversion / reshape	—	$O(in)$
<code>forward()</code> per layer	$O(out \times in)$	$O(out \times in)$
Loop over L layers	$O(L)$	$O(L)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in)$

Table 17: Big O analysis of `prediction()`.

`_forward_batch()` (NumPy only)

This function exists only in the NumPy version. Rather than looping over B samples and calling `prediction()` for each, it transposes the batch into shape (in, B) — costing $O(in \times B)$ — and passes it through all L layers. Each layer call receives a (in, B) matrix and performs the multiply $(out, in) \times (in, B)$, costing $O(out \times in \times B)$ per layer. Total: $O(L \times out \times in \times B)$.

This is the same asymptotic class as the base-Python `compute_gradients_batch()`, which loops over B samples and calls `prediction()` each time, giving $O(B \times L \times out \times in)$. The NumPy version eliminates the Python-level loop over B , performing the same arithmetic in one BLAS call.

Operation	Base Python	NumPy
Batch transpose / reshape	—	$O(in \times B)$
<code>forward()</code> per layer (batch)	—	$O(out \times in \times B)$
Loop over L layers	—	$O(L)$
Final Big O	—	$O(L \times out \times in \times B)$

Table 18: Big O analysis of `_forward_batch()` (NumPy only).

`_compute_deltas()`

This is the core backpropagation function. The loop iterates backwards through all L layers. For each layer the following steps are performed:

- `compute_activation_derivatives()`: $O(out_i)$ — one derivative per neuron in the current layer.
- One delta per output neuron: $O(out_i)$.
- **Output layer only** — scalar loss derivative: $O(1)$.

- **Hidden layers** — the weighted sum propagates deltas from the next layer backwards. Each of the out_i neurons in the current layer must sum over all out_{i+1} neurons of the next layer, giving $O(out_i \times out_{i+1})$ per layer. The element-wise multiply with the activation derivative is $O(1)$ over each of those.

Combining these steps per layer and summing over L layers:

$$O((3 \cdot out_i + out_i \times out_{i+1}) \times L) = O(L \times out \times in)$$

since out_{i+1} plays the role of in_i in the next-layer weight matrix, and lower-order terms are dropped.

In NumPy, `np.dot(next_layer.weights.T, next_layer.delta)` replaces the Python double-loop with a single BLAS call of shape $(in_i, out_{i+1}) \times (out_{i+1}, B) \rightarrow O(in_i \times out_{i+1} \times B)$ per layer. Total: $O(L \times out \times in \times B)$.

Operation	Base Python	NumPy
Activation derivatives per layer	$O(out_i)$	$O(out_i \times B)$
Output layer loss derivative	$O(1)$	$O(out_{\text{last}} \times B)$
Hidden layer weighted delta sum	$O(out_i \times out_{i+1})$	$O(in_i \times out_{i+1} \times B)$
Element-wise delta multiply	$O(out_i)$	$O(out_i \times B)$
Loop over L layers	$O(L)$	$O(L)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in \times B)$

Table 19: Big O analysis of `_compute_deltas()`.

`compute_gradients_sample()`

Computing the gradients for a single sample consists of three sequential steps. First, `prediction()` runs the forward pass at $O(L \times out \times in)$. Then `_compute_deltas()` runs the backward pass at the same cost. Finally, a gradient computation loop iterates over all L layers: the nested comprehension over out_i rows and in_i columns fills the full weight gradient matrix at $O(out_i \times in_i)$ per layer, and the bias gradient loop adds $O(out_i)$ per layer. Summing:

$$O(2 \times L \times out \times in + L) = O(L \times out \times in)$$

In NumPy, `np.dot(delta, prev_a.T)` replaces the nested comprehension with a single outer-product call of shape $(out, 1) \times (1, in)$, giving the same $O(out \times in)$ per layer.

`compute_gradients_batch()`

The function begins with length checks on the batch inputs and targets, which are $O(1)$. The next step initialises zero accumulators: one per weight gradient entry, giving $O(L \times out \times in)$ for `dweights` and $O(L \times out)$ for `dbiases`. Then gradients are computed per sample. The loop runs B times; each iteration calls `compute_gradients_sample()` at $O(L \times out \times in)$, then accumulates layer by layer at the same cost. This gives $O(B \times L \times out \times in)$ for the accumulation

Operation	Base Python	NumPy
<code>prediction()</code> — forward pass	$O(L \times out \times in)$	$O(L \times out \times in)$
<code>_compute_deltas()</code> — backward pass	$O(L \times out \times in)$	$O(L \times out \times in)$
<code>dweights</code> gradient fill per layer	$O(out \times in)$	$O(out \times in)$
<code>dbiases</code> gradient fill per layer	$O(out)$	$O(out)$
Loop over L layers	$O(L)$	$O(L)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in)$

Table 20: Big O analysis of `compute_gradients_sample()`.

phase. Finally, all accumulated entries are divided by B to obtain averages, adding another $O(L \times out \times in)$. Collecting all terms:

$$O(1 + L \cdot out \cdot in + L \cdot out + B(2L \cdot out \cdot in + L \cdot out) + L \cdot out \cdot in) = O(B \times L \times out \times in)$$

In NumPy, `_forward_batch()` and `_compute_deltas()` each process all B samples simultaneously, costing $O(L \times out \times in \times B)$ each. The gradient loop computes `np.dot(delta, prev_a.T)` with shapes $(out, B) \times (B, in) \rightarrow (out, in)$, giving $O(out \times in \times B)$ per layer. `delta.mean(axis=1)` adds $O(out \times B)$ per layer. Total: $O(L \times out \times in \times B)$. The asymptotic class is identical; NumPy again just eliminates the Python sample-loop.

Operation	Base Python	NumPy
Input length checks	$O(1)$	$O(1)$
Initialise zero accumulators	$O(L \times out \times in)$	—
<code>np.array()</code> batch conversion	—	$O(B \times in)$
Forward pass (batch)	$O(B \times L \times out \times in)$	$O(L \times out \times in \times B)$
Backward pass (batch)	$O(B \times L \times out \times in)$	$O(L \times out \times in \times B)$
Gradient accumulation / computation	$O(B \times L \times out \times in)$	$O(L \times out \times in \times B)$
Average over B	$O(L \times out \times in)$	$O(L \times out \times B)$
Final Big O	$O(B \times L \times out \times in)$	$O(L \times out \times in \times B)$

Table 21: Big O analysis of `compute_gradients_batch()`.

8.2.3 ANN Training and Evaluation

`_save_parameters_snapshot()`

This helper works by copying all weights and biases to a new structure.

In **Base Python** the function uses a list comprehension over all L layers. For each layer, `row[:]` creates a copy of each row in the weight matrix — iterating over out rows of in values gives $O(out \times in)$ per layer. The bias copy iterates over out rows of length 1, giving $O(out)$ per layer. Dominant term per layer: $O(out \times in)$. Over L layers: $O(L \times out \times in)$.

In **NumPy** `layer.weights.copy()` and `layer.biases.copy()` each allocate a new array and copy every element — $O(out \times in)$ and $O(out)$ per layer. Over L layers that makes $O(L \times out \times in)$.

Operation	Base Python	NumPy
Copy weight matrix per layer	$O(out \times in)$	$O(out \times in)$
Copy bias vector per layer	$O(out)$	$O(out)$
Loop over L layers	$O(L)$	$O(L)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in)$

Table 22: Big O analysis of `_save_parameters_snapshot()`.

`_restore_parameters_snapshot()`

This function is the inverse of the above, which means the final big O notation is the same - in **Base Python** the loop iterates over L layers. For each layer, `row[:]` copies each row of the saved weight matrix back — $O(out \times in)$ — and the bias copy costs $O(out)$. Over L layers that is $O(L \times out \times in)$.

In **NumPy**, `w.copy()` and `b.copy()` copy all values back to the fresh arrays of the layer — $O(out \times in)$ and $O(out)$ per layer. Over L layers it is $O(L \times out \times in)$.

Operation	Base Python	NumPy
Restore weight matrix per layer	$O(out \times in)$	$O(out \times in)$
Restore bias vector per layer	$O(out)$	$O(out)$
Loop over L layers	$O(L)$	$O(L)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in)$

Table 23: Big O analysis of `_restore_parameters_snapshot()`.

`compute_loss()`

This function is called once per epoch during training for both the training and validation set. It iterates over all samples in the input batch. For each sample it calls `prediction()` at $O(L \times out \times in)$, then sums over the out_{last} output neurons to compute the per-sample loss — $O(out_{last})$. For binary classification with a single output neuron this is $O(1)$. Iterating over M samples (where M is the size of the set passed in, either N for training or the validation set size):

$$O(M \times L \times out \times in)$$

NumPy has additional type and shape checks, that are all $O(1)$ comparisons. `self.prediction(X)` runs a single batched forward pass over all M samples simultaneously — $O(L \times out \times in \times M)$. `np.mean(loss_func(Y, Y_pred))` applies the loss element-wise and reduces — $O(out \times M)$. Dominant term is $O(L \times out \times in \times M)$.

Both versions therefore have the same asymptotic class. When called with the full training set ($M = N$) the cost is $O(N \times L \times out \times in)$.

`train()`

Operation	Base Python	NumPy
Type / shape checks	$O(1)$	$O(1)$
<code>prediction()</code> per sample	$O(L \times out \times in)$	—
Batched <code>prediction(X)</code>	—	$O(L \times out \times in \times M)$
Per-sample loss sum	$O(out_{last})$	—
<code>np.mean()</code> over output	—	$O(out \times M)$
Loop over M samples	$O(M)$	—
Final Big O	$O(M \times L \times out \times in)$	$O(L \times out \times in \times M)$

Table 24: Big O analysis of `compute_loss()`. M is the number of samples in the set passed in.

Training introduces variables E (maximum number of epochs), E^* (actual epochs run, where $E^* \leq E$), and N (total training samples). The function takes one full pass over the dataset per epoch, so all per-epoch costs are multiplied by E^* . Then there is also a validation set evaluation each epoch, early stopping based on a patience parameter P , and a parameter snapshot restoration at the end. While these additions add constant-factor work, they do not change the asymptotic class.

At the start of each epoch, the learning-rate decay check is $O(1)$. Shuffling N training indices costs $O(N)$. In **Base Python**, this involves building an index list and constructing shuffled lists via comprehension. In **NumPy**, `self.rng.permutation` generates a permuted array in $O(N)$, while fancy indexing `X[indices]` reorders N columns at $O(N \times in)$ and $O(N \times out)$; these are dominated by the training cost.

The mini-batch loop runs N/B times per epoch. Each iteration calls `compute_gradients_batch()` at $O(B \times L \times out \times in)$, followed by a parameter update over L layers at $O(L \times out \times in)$. Over all batches per epoch:

$$\frac{N}{B} \times O(B \times L \times out \times in) = O(N \times L \times out \times in)$$

After each epoch, `compute_loss()` is called for the training set ($M = N$) and the validation set ($M = N_{val}$), costing $O(M \times L \times out \times in)$ per call. This matches the dominant per-epoch training term. If verbose output is enabled, loss re-evaluation adds the same complexity. If validation loss improves, `_save_parameters_snapshot()` is called at $O(L \times out \times in)$, which is not dominant. At the end of training, `_restore_parameters_snapshot()` is called once, also at $O(L \times out \times in)$.

All per-epoch costs collapse to $O(N \times L \times out \times in)$. Multiplying by E^* epochs gives the final result:

$$O(E^* \times N \times L \times out \times in)$$

In the uniform-width case (W), this simplifies to $O(E^* \times N \times L \times W^2)$. Early stopping may reduce the constant E^* , but the asymptotic class remains identical for both Base Python and NumPy versions.

`save_model()`

Operation	Base Python	NumPy
Epoch loop (E^* iterations)	$O(E^*)$	$O(E^*)$
LR decay check	$O(1)$	$O(1)$
Shuffle N indices	$O(N)$	$O(N)$
Mini-batch loop (N/B iterations)	$O(N/B)$	$O(N/B)$
<code>compute_gradients_batch()</code> per batch	$O(B \cdot L \cdot out \cdot in)$	$O(B \cdot L \cdot out \cdot in)$
Parameter update per batch	$O(L \cdot out \cdot in)$	$O(L \cdot out \cdot in)$
<code>compute_loss()</code> (Train + Val + Verbose)	$O(N \cdot L \cdot out \cdot in)$	$O(N \cdot L \cdot out \cdot in)$
<code>_save_parameters_snapshot()</code> (if improved)	$O(L \cdot out \cdot in)$	$O(L \cdot out \cdot in)$
<code>_restore_parameters_snapshot()</code> (once)	$O(L \cdot out \cdot in)$	$O(L \cdot out \cdot in)$
Per-epoch cost	$O(N \cdot L \cdot out \cdot in)$	$O(N \cdot L \cdot out \cdot in)$
Final Big O	$O(E^* \cdot N \cdot L \cdot out \cdot in)$	$O(E^* \cdot N \cdot L \cdot out \cdot in)$

Table 25: Unified Big O analysis of `train()` with validation and early stopping.

For saving the model in a file, type checks and path operations (`Path.mkdir`, `open`) are $O(1)$. Writing the metadata header lines is also $O(1)$ since the number of hyperparameters is fixed.

The dominant cost is writing the parameters. For each of the L layers, the weight matrix of shape (out, in) is written row by row — $O(out \times in)$. Then the bias vector of length out is written value by value — $O(out)$. Iterating over L layers: $O(L \times out \times in)$.

The two versions are structurally identical here. The only difference is that the NumPy version reads `layer.weights.shape` directly ($O(1)$) rather than calling `len()`, and iterates `layer.biases[:, 0]` instead of nested lists, but both give the same Big O.

Operation	Base Python	NumPy
Type checks and path setup	$O(1)$	$O(1)$
Write metadata headers	$O(1)$	$O(1)$
Write weight matrix per layer	$O(out \times in)$	$O(out \times in)$
Write bias vector per layer	$O(out)$	$O(out)$
Loop over L layers	$O(L)$	$O(L)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in)$

Table 26: Big O analysis of `save_model()`.

`load_model()`

For reconstructing the model with the saved parameters, the first pass reads all F lines of the file into memory and separates header lines from data lines with $O(F)$. Parsing each header is $O(1)$. Building the architecture list calls `int()` once per layer entry — $O(L)$. Constructing the ANN object calls `_build_ANN()`, which has $O(L \times out \times in)$.

The second pass walks through all F lines again to load weights and biases. For each layer it reads out weight rows of in values, giving $O(out \times in)$ per layer, and out bias values that are

$O(out)$ per layer. The string-to-float conversion `float(v)` is $O(1)$ per value. Over L layers: $O(L \times out \times in)$.

In **base Python** weights are stored as a list of lists directly. **NumPy**'s `np.array(rows)` and `np.array(values).reshape(-1, 1)` convert the parsed lists into arrays — each costs $O(out \times in)$ and $O(out)$ per layer respectively, which does not change the dominant term.

The total number of lines F satisfies $F = O(L \times out \times in)$ since the file contains exactly one line per weight value plus a fixed number of header lines. Both passes are therefore bounded by $O(L \times out \times in)$.

Operation	Base Python	NumPy
Read all F lines into memory	$O(F)$	$O(F)$
Parse header dict	$O(1)$	$O(1)$
Build ANN (<code>_build_ANN()</code>)	$O(L \times out \times in)$	$O(L \times out \times in)$
Parse weight rows per layer	$O(out \times in)$	$O(out \times in)$
<code>np.array()</code> conversion per layer	—	$O(out \times in)$
Parse bias values per layer	$O(out)$	$O(out)$
Loop over L layers	$O(L)$	$O(L)$
Final Big O	$O(L \times out \times in)$	$O(L \times out \times in)$

Table 27: Big O analysis of `load_model()`. $F = O(L \times out \times in)$ is the total number of lines in the saved model file.

8.2.4 Overall ANN Complexity

The dominant cost across the entire ANN class is the training loop. Every epoch must perform at least one forward and one backward pass over all N samples through all L layers, and each layer pass requires $out \times in$ multiply-add operations. This is the unavoidable cost of a matrix-vector product, and no implementation can reduce this without changing the algorithm itself.

The addition of early stopping, model persistence, and explicit loss tracking does not change the asymptotic complexity of the ANN. All new functions — `_save_parameters_snapshot()`, `_restore_parameters_snapshot()`, `compute_loss()`, `save_model()`, and `load_model()` — are bounded by $O(L \times out \times in)$, which is smaller than the per-epoch training cost by a factor of N .

The overall complexity of the ANN therefore remains:

$$O(E^* \times N \times L \times out \times in)$$

where $E^* \leq E$ is the number of epochs actually completed before early stopping. In the uniform-width case, this is expressed as $O(E^* \times N \times L \times W^2)$. Both the base Python and NumPy implementations share this class exactly; NumPy's advantage is a substantially smaller constant factor, not a lower asymptotic complexity.

8.3 Big O of Evaluation

The evaluation module computes classification metrics on a dataset of M samples. The following analysis uses M for the number of samples in the set being evaluated (training, validation, or test). All functions call `ann.prediction()` internally; as established in the ANN analysis, one prediction over M samples costs $O(M \times L \times out \times in)$ in the base Python version (loop over samples) and the same class in the NumPy version (single batched forward pass). This is the dominant term in every evaluation function.

`accuracy()`

In **Base Python**, a `for` loop iterates over all M samples. Each iteration calls `ann.prediction(x)` at $O(L \times out \times in)$, applies the threshold comparison in $O(1)$, and increments a counter in $O(1)$. The final division is $O(1)$. Total is therefore $O(M \times L \times out \times in)$.

NumPy's `_to_batch()` normalises array shapes — iterating over $M \times F$ elements to transpose or reshape, giving $O(M \times F)$. `ann.prediction(X)` runs a batched forward pass over all M samples at $O(L \times out \times in \times M)$. The threshold comparison and `np.mean()` each visit M elements — $O(M)$. Dominant term: $O(M \times L \times out \times in)$.

Operation	Base Python	NumPy
<code>_to_batch()</code> shape normalisation	—	$O(M \times F)$
<code>prediction()</code> per sample / batch	$O(L \times out \times in)$	$O(L \times out \times in \times M)$
Threshold comparison	$O(1)$ per sample	$O(M)$
Loop over M samples	$O(M)$	—
<code>np.mean()</code>	—	$O(M)$
Final Big O	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$

Table 28: Big O analysis of `accuracy()`.

`confusion_matrix()`

Base Python: The structure is identical to `accuracy()` — a loop over M samples, each calling `ann.prediction(x)` at $O(L \times out \times in)$, with $O(1)$ comparisons and counter increments per sample. Total: $O(M \times L \times out \times in)$.

NumPy: After `_to_batch()` at $O(M \times F)$, a single batched `ann.prediction(X)` runs at $O(M \times L \times out \times in)$. The four boolean mask operations (`labels == 0`, `preds == 0`, etc.) and `np.sum()` each iterate over M elements — $O(M)$ each, four times. The `.flatten()` calls on the prediction and label arrays are also $O(M)$. All dominated by the forward pass.

`classification_metrics()`

This function calls `confusion_matrix()` at $O(M \times L \times out \times in)$, then computes precision, recall, specificity, F1, and accuracy from the four scalar counts `tn`, `fp`, `fn`, `tp`. Each of these five metric computations involves a fixed number of arithmetic operations on scalars — $O(1)$ each.

Operation	Base Python	NumPy
<code>_to_batch()</code> shape normalisation	—	$O(M \times F)$
<code>prediction()</code> per sample / batch	$O(L \times out \times in)$	$O(L \times out \times in \times M)$
<code>.flatten()</code> on predictions and labels	—	$O(M)$
Boolean masks + <code>np.sum()</code> ($\times 4$)	$O(1)$ per sample	$O(M)$ each
Loop over M samples	$O(M)$	—
Final Big O	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$

Table 29: Big O analysis of `confusion_matrix()`.

The return of a dictionary is $O(1)$. The dominant cost is the `confusion_matrix()` call. Both versions are identical in structure here; the version-specific differences are already accounted for inside `confusion_matrix()`.

Operation	Base Python	NumPy
<code>confusion_matrix()</code> call	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$
Metric arithmetic (5 scalars)	$O(1)$	$O(1)$
Final Big O	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$

Table 30: Big O analysis of `classification_metrics()`.

`evaluate()`

`evaluate()` calls `ann.compute_loss()` at $O(M \times L \times out \times in)$ and `classification_metrics()` at the same cost, then calls `print_confusion_matrix()` which internally calls `confusion_matrix()` again at $O(M \times L \times out \times in)$. All print statements are $O(1)$. The three calls to functions of equal asymptotic cost sum to $O(M \times L \times out \times in)$.

In the **NumPy version**, `_to_batch()` is called once at the top of `evaluate()` before any other call — $O(M \times F)$ — and the normalised arrays are passed downstream, avoiding repeated normalisation inside each sub-call.

Operation	Base Python	NumPy
<code>_to_batch()</code> (once, at top)	—	$O(M \times F)$
<code>compute_loss()</code>	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$
<code>classification_metrics()</code>	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$
<code>print_confusion_matrix()</code>	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$
Print statements	$O(1)$	$O(1)$
Final Big O	$O(M \times L \times out \times in)$	$O(M \times L \times out \times in)$

Table 31: Big O analysis of `evaluate()`.

`report_results()`

This is the lightweight alternative to `evaluate()`. It calls `accuracy()` twice — once for the training set ($M = N$) and once for the validation set ($M = N_{\text{val}}$) — and prints two lines. Each `accuracy()` call costs $O(M \times L \times \text{out} \times \text{in})$. The two calls together are bounded by $O(N \times L \times \text{out} \times \text{in})$ since $N_{\text{val}} \leq N$. Print statements are $O(1)$.

Operation	Base Python	NumPy
<code>accuracy()</code> on training set	$O(N \times L \times \text{out} \times \text{in})$	$O(N \times L \times \text{out} \times \text{in})$
<code>accuracy()</code> on validation set	$O(N_{\text{val}} \times L \times \text{out} \times \text{in})$	$O(N_{\text{val}} \times L \times \text{out} \times \text{in})$
Print statements	$O(1)$	$O(1)$
Final Big O	$O(N \times L \times \text{out} \times \text{in})$	$O(N \times L \times \text{out} \times \text{in})$

Table 32: Big O analysis of `report_results()`.

8.3.1 Overall Evaluation Complexity

Every evaluation function is dominated by the cost of running predictions over the dataset. Since all functions ultimately call `ann.prediction()` once per sample (base Python) or once for the full batch (NumPy), and since `classification_metrics()` and the metric arithmetic operate on a fixed number of scalar values after that, the asymptotic cost of the entire evaluation module is:

$$O(M \times L \times \text{out} \times \text{in})$$

where M is the number of samples in the set being evaluated. This is smaller than the training cost by a factor of E^* , confirming that evaluation adds no meaningful asymptotic overhead to the overall programme. Both versions share this class; the NumPy version eliminates the per-sample Python loop via batched prediction, but the scaling behaviour is identical.

8.4 Big O of Data Parsing

The data parsing module reads `.howlin` files and converts each line into the format expected by the ANN. The following analysis uses S for the total number of lines in the file (i.e. the dataset size), and F for the number of features per sample (27 in this project). The functions are the same in both versions at the algorithmic level; differences are noted where they occur.

`parse_line()`

The function calls `line.strip().split()`, which scans all characters in the line once — $O(F)$ since each line contains F feature values plus one label. The length check is $O(1)$. The list comprehension that converts the first F entries to `float` iterates over F values — $O(F)$. Parsing the final entry as `int` is $O(1)$. Both versions are identical here.

`iterate_input()`

Operation	Base Python	NumPy
<code>strip().split()</code>	$O(F)$	$O(F)$
Length check	$O(1)$	$O(1)$
Float conversion (F values)	$O(F)$	$O(F)$
Int conversion (label)	$O(1)$	$O(1)$
Final Big O	$O(F)$	$O(F)$

Table 33: Big O analysis of `parse_line()`.

The function is a generator. The boundary checks on `start` and `end` are $O(1)$. The `for` loop reads lines one at a time and yields after calling `parse_line()` on each. If the full file of S lines is consumed, `parse_line()` is called S times at $O(F)$ each. The `continue/break` guards for `start` and `end` are $O(1)$ per line. Total: $O(S \times F)$. Both versions are structurally identical.

Operation	Base Python	NumPy
Boundary checks	$O(1)$	$O(1)$
Loop over S lines	$O(S)$	$O(S)$
<code>parse_line()</code> per line	$O(F)$	$O(F)$
Final Big O	$O(S \times F)$	$O(S \times F)$

Table 34: Big O analysis of `iterate_input()`.

`parse_input()`

Both versions call `iterate_input()` to stream all S samples, costing $O(S \times F)$. The divergence is in how the results are stored.

Base Python: Each sample is immediately reshaped into a $(F, 1)$ column vector via a list comprehension — $O(F)$ per sample. Over S samples the total reshaping cost is $O(S \times F)$, which does not change the dominant term.

NumPy: Flat lists are accumulated without reshaping, then converted in bulk at the end. `np.array(input_list, dtype=float32)` allocates a contiguous (S, F) buffer and copies every element — $O(S \times F)$. `np.array(label_list, dtype=int32)` allocates an $(S, 1)$ buffer — $O(S)$. Dominant term: $O(S \times F)$.

Operation	Base Python	NumPy
<code>iterate_input()</code> (stream S lines)	$O(S \times F)$	$O(S \times F)$
Reshape each sample to column vector	$O(F)$ per sample = $O(S \times F)$	—
<code>np.array()</code> bulk conversion	—	$O(S \times F)$
Final Big O	$O(S \times F)$	$O(S \times F)$

Table 35: Big O analysis of `parse_input()`.

`load_and_split()` (base Python only)

This generator exists only in the base Python version. It first calls `parse_input()` to load the entire file into memory — $O(S \times F)$. Zipping inputs and labels into a list of tuples iterates over S pairs — $O(S)$. Shuffling with `random.shuffle()` is $O(S)$ (Fisher-Yates in-place). The `for` loop then yields $\lceil S/B \rceil$ batches, with each batch iteration performing a slice — $O(B)$ — and an `unzip` via `zip(*batch)` — $O(B)$. Over all batches this is $O(S)$ total. The dominant cost is therefore the initial `parse_input()` call.

Operation	Base Python	NumPy
<code>parse_input()</code> (load full file)	$O(S \times F)$	—
Zip into list of tuples	$O(S)$	—
<code>random.shuffle()</code>	$O(S)$	—
Yield $\lceil S/B \rceil$ batches (slice + unzip)	$O(S)$	—
Final Big O	$O(S \times F)$	—

Table 36: Big O analysis of `load_and_split()` (base Python only).

8.4.1 Overall Data Parsing Complexity

All parsing functions are bounded by $O(S \times F)$, since every non-trivial operation must visit every feature value of every sample at least once. $F = 27$ is a fixed constant in this project, so the complexity reduces to $O(S)$ in practice — linear in the number of samples. Both versions share this class; the NumPy version defers memory allocation to a single `np.array()` call rather than constructing nested lists, but does not reduce the asymptotic cost.

8.5 Big O of main

The `main` function orchestrates data loading, preprocessing, training, and evaluation. It does not implement any new algorithmic logic itself; it assembles calls to the functions analysed above. The analysis here therefore identifies which calls dominate and confirms that `main` does not introduce any additional asymptotic cost.

The following additional symbol is introduced:

- N_{val} — number of validation samples ($\approx 0.15 \times S$ after the 85/15 split). $N = S - N_{\text{val}}$ is the number of training samples.

Data loading and validation split

`parse_input()` loads the full file — $O(S \times F)$. The 85/15 validation split builds a shuffled index list over S entries — $O(S)$ — then constructs two sublists via list comprehensions, each iterating over S indices — $O(S)$ total. Class counts (`ones`, `zeros`) iterate over the label list once — $O(S)$. Dominant term: $O(S \times F)$, which reduces to $O(S)$ since $F = 27$ is constant.

Operation	Base Python	NumPy
<code>parse_input()</code>	$O(S \times F)$	$O(S \times F)$
Shuffle index list	$O(S)$	$O(S)$
Train/val split (list comprehensions)	$O(S)$	$O(S)$
Class count	$O(S)$	$O(S)$
Final Big O	$O(S \times F)$	$O(S \times F)$

Table 37: Big O analysis of data loading and validation split in `main()`.

Oversampling

Finding class indices iterates over the training labels once — $O(N)$. `random.choices()` samples $diff$ indices with replacement, where $diff \leq N$ — $O(N)$ in the worst case. Extending `X_train` and `Y_train` with the duplicated samples iterates over $diff$ entries — $O(diff \times F) \leq O(N \times F)$. The final shuffle using `zip` and `random.shuffle` over the combined list is $O(N)$. Dominant term: $O(N \times F)$, again $O(N)$ for fixed F .

Operation	Base Python	NumPy
Find class indices	$O(N)$	$O(N)$
<code>random.choices()</code>	$O(N)$	$O(N)$
Extend training lists	$O(N \times F)$	$O(N \times F)$
Shuffle combined list	$O(N)$	$O(N)$
Final Big O	$O(N \times F)$	$O(N \times F)$

Table 38: Big O analysis of oversampling in `main()`.

Training

The call to `ann.train()` dominates all other steps in `main`. As established in the ANN analysis:

$$O(E^* \times N \times L \times out \times in)$$

No additional cost is introduced by `main` here.

Evaluation

After training, `evaluate()` is called on the training set, the validation set, and the test set. Each call runs `compute_loss()` and `classification_metrics()`, which both iterate over all samples in the set passed in. As established in the evaluation analysis, one `evaluate()` call on a set of M samples costs:

$$O(M \times L \times out \times in)$$

For the training set $M = N$, for validation $M = N_{\text{val}}$, and for the test set $M = N_{\text{test}}$. Since N_{val} and N_{test} are at most comparable to N , all three calls are bounded by $O(N \times L \times out \times in)$, which is smaller than the training cost by a factor of E^* .

Operation	Base Python	NumPy
<code>evaluate()</code> on training set	$O(N \cdot L \cdot out \cdot in)$	$O(N \cdot L \cdot out \cdot in)$
<code>evaluate()</code> on validation set	$O(N_{val} \cdot L \cdot out \cdot in)$	$O(N_{val} \cdot L \cdot out \cdot in)$
<code>evaluate()</code> on test set	$O(N_{test} \cdot L \cdot out \cdot in)$	$O(N_{test} \cdot L \cdot out \cdot in)$
Final Big O	$O(N \cdot L \cdot out \cdot in)$	$O(N \cdot L \cdot out \cdot in)$

Table 39: Big O analysis of post-training evaluation in `main()`.

8.5.1 Overall main Complexity

Collecting all steps, the dominant cost in `main` is the call to `ann.train()`. All preprocessing steps — parsing, splitting, oversampling — are $O(S \times F) = O(N)$ for fixed F , and all post-training evaluation calls are $O(N \times L \times out \times in)$, smaller than training by a factor of E^* . The overall complexity of `main` is therefore:

$$O(E^* \times N \times L \times out \times in)$$

This is identical to the complexity of `train()` alone, confirming that the orchestration code in `main` adds no asymptotic overhead. Both the base Python and NumPy versions share this class; the NumPy version benefits from a smaller constant factor through vectorised operations, but the scaling behaviour with respect to E^* , N , L , and layer width is the same in both.

9 Unit Test

A unit-test suite was developed using `pytest` to verify the correctness of the network. Because the project provides two parallel implementations of the same model, a base-Python version using nested lists and a NumPy version using vectorised array operations, the test suite is written in two parallel branches as well, one for each implementation. The two branches share the same overall structure and cover the same methods, differing only in the data types they pass in (lists of lists versus `numpy.ndarray`) and in the assertions they use to compare outputs. This parallel design lets both implementations be validated against the same correctness criteria, ensuring they behave equivalently despite their different internal representations.

The tests are organised one file per method, mirroring the structure of the source code. They are grouped into two layers: tests targeting the `ANN_Layer` class (the building block) and tests targeting the `ANN` class (the full network).

9.1 Coverage

At the **layer level**, the suite covers the core building blocks:

- the constructor (`__init__`)
- the `weights` and `biases` property setters
- `initialize_weights_bias`
- the forward pass
- `compute_activation_derivatives`
- `update_parameters`

At the **network level**, it covers:

- the constructor (`__init__`) and `_build_ANN`
- `prediction` – the forward pass through the whole network
- `_compute_deltas` – the backward pass
- `compute_gradients_sample`
- `compute_loss`
- `save_model` and `load_model`

9.2 Types of evaluations

Each method is tested along three complementary axes, chosen so that together they catch the three most common categories of bug: incorrect computations, missing input validation, and broken algorithmic invariants.

- **Happy-path tests.** Feed valid inputs and check that the output has the right shape and the right values. Expected values are computed by hand for small examples (e.g. a 2×2 weight matrix and a known input), or compared against a reference formula for deterministic operations such as activation functions and weight initialisation statistics.
- **Type and value validation tests.** Feed deliberately malformed inputs as wrong type, wrong shape, empty arrays, mismatched dimensions, out-of-range values and check that the method raises the expected `TypeError` or `ValueError` with a meaningful message. These tests guard the public API against silent failures.
- **Consistency and sanity tests.** Verify invariants that must hold regardless of the specific input values. Examples include:
 - A batched forward pass produces the same result as stacking individual single-sample passes.
 - Binary cross-entropy loss is always non-negative.
 - Sigmoid outputs always lie in $(0, 1)$.
 - A small step in the negative-gradient direction reduces the loss (a numerical-gradient sanity check).
 - Training on a trivially separable dataset drives the loss below a small threshold within a few epochs.

9.3 Round-trip testing for I/O

The save/load functionality is verified with a single round-trip test: a trained model is written to disk with `save_model()`, reloaded with `load_model()`, and the reloaded network is required to produce predictions numerically identical to the original (within floating-point precision). This single test catches almost any bug in either method, since a corruption in the file format, the parsing logic, or the parameter assignment will manifest as a prediction mismatch.

9.4 Testing gaps

The test suite has noticeable gaps. The training loop itself is not unit-tested: it is exercised only indirectly, through end-to-end experiments, so a regression in the optimiser, the learning-rate schedule, or the early-stopping logic would only surface when overall training results changed. Batch gradient computation is similarly untested in isolation, despite being a more error-prone path than the per-sample case it generalises. Model saving and loading have basic round-trip tests but no coverage of failure modes such as malformed files, missing fields, or schema mismatches between the saving and loading routines.

10 Conclusion

The aim of this project was to implement a feedforward neural network with backpropagation from scratch, in two different ways: one in pure Python using only the standard library, and one using NumPy. The focus was on the implementation itself rather than on the results of the binary classification task, which served primarily as a vehicle for testing that the implementations worked correctly.

The comparison between the two implementations confirmed the expected outcome: the NumPy implementation is substantially more time-efficient than the pure Python version. By replacing nested lists and explicit loops with vectorised array operations, NumPy leverages optimised, low-level routines that scale far better with both network size and dataset size. The pure Python implementation, while functionally equivalent, becomes impractical for anything beyond small architectures and short training runs. This reinforces a broader lesson: in numerical computing, the choice of data structures and the use of vectorisation matter as much as algorithmic correctness.

Beyond raw performance, the two implementations also differed in code clarity. The pure Python version makes every operation in the forward and backward passes explicit, which proved valuable as a learning exercise, each step of the chain rule can be traced through the code. The NumPy version, by contrast, hides much of that detail inside matrix operations, trading transparency for conciseness and speed. Both views are useful, and having implemented both gave a deeper understanding of what vectorisation actually does under the hood.

10.1 Methodological caveats and known limitations

Several caveats and known issues are worth recording for transparency, both as a record of what we noticed during development and as natural starting points for future work.

The most prominent methodological caveat concerns the random number generators. The two implementations rely on different RNGs: Python's `random` for the pure-Python version and `numpy.random` for the NumPy version. Even with identical seeds, the two generators produce different sequences, so the train/validation splits, the oversampling of the minority class, and the weight initialisations are not directly comparable across implementations. A controlled comparison that isolates the implementation difference would require feeding both versions the same underlying random sequence, under those conditions we would expect the predictions to converge to near-identical values, with any residual differences attributable to floating-point arithmetic. As the code currently stands, however, each implementation is hard-wired to its own generator and exposes no mechanism for injecting the other, so realising such a comparison would require a small amount of refactoring before it could be carried out. On a more positive note, within each implementation the RNG is held as an instance attribute seeded in `__init__` and used for both weight initialisation and the train/validation split, which makes individual training runs reproducible from a single seed.

The two codebases also have a few consistency issues worth flagging. The most awkward sits inside the base-Python version itself: most quantities (weights, biases, pre-activations, activations) follow a column-vector convention and are stored as nested lists (`[[3],[5]]`), whereas the activation derivatives are kept as a flat one-dimensional list (`[1,1]`). This breaks the otherwise uniform shape convention of the implementation and forces small reshape-style

adjustments every time the derivatives have to interact with the other quantities. The NumPy version is internally consistent on this point, with its derivatives stored as two-dimensional arrays alongside the rest of the layer state. Instead, its inconsistency lies on its public-facing surface, where some functions accept inputs in slightly different formats. Equivalent functions across the two implementations also sometimes differ in what they validate, so a value that would be rejected in one version may be quietly accepted in the other. None of this affects correctness, but it makes the two codebases harder to compare side by side and harder to maintain in lockstep.

There are also a handful of issues internal to one or other implementation. In the pure-Python version, the per-sample intermediates used during batch backpropagation, the pre-activations z , the activations a , and the activation derivatives, are overwritten by each subsequent sample in the batch rather than being explicitly cleared and re-accumulated. The current code handles this implicitly through the order of operations, but a more defensive design would reset these buffers between samples. Separately, partway through development we introduced property setters for the layer parameters; as a result some input checks are now performed both at the call site and inside the setter, and these duplicate checks could be consolidated.

Several rough edges remain on the input-handling and evaluation side. Batch inputs are never explicitly validated, and what validation does exist is scattered across the training and testing paths rather than centralised. Model loading similarly lacks error handling: a malformed or missing file produces an unhelpful low-level exception rather than a clear diagnostic. Data shuffling sits outside the input parser; folding it into the parsing stage would both simplify the training loop and remove a further point of divergence between the two implementations. Finally, the `evaluate` module of both implementations contains inefficiencies that we did not address, it produces correct results but does more work than it needs to.

10.2 Future work

Several directions could extend this work:

- **Unifying the random number generator** across both implementations to enable a fully controlled comparison of numerical outputs, isolating the effect of the implementation from the effect of randomness.
- **Benchmarking systematically** across a range of network sizes, batch sizes, and dataset sizes to quantify the speed-up factor of NumPy over pure Python as a function of problem scale.
- **Adding more rigorous evaluation** such as k -fold cross-validation, which would give a more reliable estimate of generalisation performance than a single train/validation split.
- **Resolving the consistency and robustness issues** listed above: unifying the data structures used for derivatives, consolidating input validation, adding error handling on model loading, refactoring the `evaluate` module, and moving data shuffling into the input-parsing stage.

Overall, the project achieved its goal: producing two working, well-tested implementations of a neural network with backpropagation, and characterising the practical differences between

them. The pure Python version stands as a transparent reference implementation, and the NumPy version as a more practical baseline for any future extension of the work.

11 Declaration of use of generative AI

Generative AI tools were used during the preparation of this project. Specifically, Claude AI was used to support both the writing and coding components of the work.

For the report, Claude was used to proofread drafts and improve clarity, sentence structure, and overall readability. The tool was used throughout the writing process, and suggested rephrasings were incorporated only when they preserved our original meaning and helped communicate the points more effectively to the reader. All ideas, arguments, and conclusions presented in the report are our own.

For the code, Claude was used to assist with debugging, to confirm the correctness of implementations. When the tool was used to help resolve errors, its suggestions were reviewed carefully before being incorporated, and were used as guidance to locate and understand the underlying issue ourselves. The design of the algorithms and the overall structure of the program remain our own work.

In all cases, generative AI was treated as a supporting tool rather than a substitute for our own analysis, and outputs were critically evaluated before being used.

References

- [1] Vrs Jhalia and Tripti Swarnkar. A critical review on the application of artificial neural network in bioinformatics, 2021.
- [2] Wikipedia contributors. Artificial neural network. https://en.wikipedia.org/wiki/Artificial_neural_network, 2026. Accessed: 2026-04-16.
- [3] DTU. *Introduction to Machine Learning and Data Mining – Lecture Notes*, 2023. Fall 2023, Version 1.0.
- [4] Anton A. Komar, editor. *Single Nucleotide Polymorphisms: Methods and Protocols*, volume 578 of *Methods in Molecular Biology*. Humana Press, Totowa, NJ, 2 edition, 2009.
- [5] Larisa Bešić, Imer Muhović, Adna Ašić, Aida Čatić, Lejla Gurbeta, and Almir Badnjevic. Application of neural networks to the prediction of a phenotypic trait of Pacific lampreys based on single nucleotide polymorphism (SNP) genetic markers. *Biomedical Research and Clinical Practice*, 2, 2017.
- [6] Grant Sanderson. Neural networks. <https://www.3blue1brown.com/topics/neural-networks>, 2017. 3Blue1Brown. Accessed: 2026-04-16.
- [7] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In Yee Whye Teh and Mike Titterton, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256. PMLR, 2010.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.
- [9] Wikipedia contributors. Backpropagation. <https://en.wikipedia.org/wiki/Backpropagation>, 2026. Accessed: 2026-04-16.
- [10] Samson Zhang. Building a neural network from scratch. https://www.youtube.com/watch?v=sIX_9n-1UbM, 2022. Accessed: 2026-04-16.
- [11] Marcus Zafar et al. Pure Python vs NumPy vs TensorFlow performance comparison, 2023.
- [12] Artur Cepuc. Boosting Python performance: Exploring the power of vectorization vs loops, 2024.
- [13] John Lu. Why vectorized operations in NumPy are faster?, 2024.